

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Universitat Politècnica
de València

Departamento de Sistemas Informáticos y Computación

DSIC
DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

Trabajo Fin de Máster

Tipos cociente en un lenguaje dependientemente tipado

Autor(a): Luis Vegas
Director(a): Santiago Escobar

Valencia, septiembre de 2026

Este Trabajo Fin de Máster se ha depositado en el Departamento de Sistemas Informàticos y Computación de la Universitat Politècnica de València para su defensa.

Trabajo Fin de Máster
Máster Universitario en Ingeniería y Tecnología de Sistemas Software

Título: Tipos Cociente en un Lenguaje Dependientemente Tipado

Septiembre, 2026

Autor(a): Luis Vegas
Director(a): Santiago Escobar
Departamento de Sistemas Informáticos y Computación
Universitat Politècnica de València

"What I cannot build. I do not understand."

— Richard Feynman

"Be ashamed to die until you have won some victory for humanity."

— Horace Mann

Todo el trabajo presentado, incluyendo el análisis, la interpretación de los resultados y las conclusiones, ha sido realizado por mí, salvo aquellas fuentes y contribuciones que se encuentran debidamente citadas o reconocidas en el presente documento.

Resumen

Este trabajo presenta el diseño e implementación de un prototipo de lenguaje de programación con tipos dependientes, extendido con tipos cociente. Los tipos dependientes permiten expresar propiedades de los programas directamente en sus tipos, mientras que los tipos cociente permiten identificar elementos de un tipo que deberían considerarse iguales según una relación de equivalencia dada, garantizando así, que las funciones definidas sobre ellos sean coherentes con dicha relación. La formalización de tipos cociente sobre sistemas de tipos dependientes mediante lenguajes basados en la reescritura, como Maude, ha recibido poca atención a lo largo de los años, y con el presente trabajo se pretende cubrir esta brecha. Adicional a ello, se pretende demostrar que Maude es sumamente eficaz para la formalización, exploración y estudio de la teoría de tipos desde la perspectiva de la reescritura de términos. Como contribuciones principales, se formaliza la sintaxis, semántica y un algoritmo de verificación de tipos bidireccional de manera ecuacional, tratando las reglas de tipado del sistema como una tripla $\mathcal{R} = (\Sigma, E, R)$. Asimismo, el cómputo sobre las clases del cociente se realiza por reducción de términos aplicando una regla de cómputo, que preserva la canonicidad del cálculo. Los resultados del presente trabajo se validan mediante la formalización de los números enteros $\mathbb{Z}$ como pares de naturales cocientados y los multiconjuntos bajo relación de permutación, demostrando así, la consistencia del sistema ante elecciones de representantes equivalentes. El trabajo incluye la definición formal del sistema de tipos, su semántica operacional, justificación de cada decisión de diseño tomada, junto con su implementación completa en Maude.

Palabras clave: tipos dependientes, tipos cociente, teoría de tipos, Maude, lógica de reescritura, reescritura de términos, lambda cálculo.

Abstract

This master's thesis presents the design and implementation of a prototype programming language with dependent types, extended with quotient types. Dependent types allow properties of programs to be expressed directly in their types, while quotient types make it possible to identify elements of a type that should be considered equal according to a given equivalence relation, thereby ensuring that functions defined on them are consistent with that relation. The formalization of quotient types over dependent type systems using rewriting-based languages, such as Maude, has received little attention over the years, and this work aims to fill this gap. In addition, this work also aims to demonstrate that Maude is highly effective for the formalization, exploration, and study of type theory from the perspective of term rewriting. As its main contributions, the syntax, semantics, and a bidirectional type-checking algorithm are formalized in an equational manner, treating the system's typing rules as a rewrite theory $\mathcal{R} = (\Sigma, E, R)$. Furthermore, computation over quotient classes is performed by term reduction using a computation rule that preserves the canonicity of the system. The results of this work are validated through the formalization of the integers $\mathbb{Z}$ as quotient pairs of natural numbers and multisets under a permutation relation, thereby demonstrating the consistency of the system under different choices of equivalent representatives. The work includes the formal definition of the type system, its operational semantics, a justification for each design decision made, along with its complete Maude implementation.

Keywords: dependent types, quotient types, type theory, Maude, rewriting logic, term rewriting, lambda calculus.

Tabla de contenidos

1. Introducción **v**

1.1. Motivación v

1.2. Objetivos vi

1.3. Objetivos específicos vi

1.4. Estructura de la tesis vi

2. El cálculo lambda y sus extensiones **1**

2.1. El cálculo lambda 1

2.1.1. Booleanos 2

2.1.2. β-reducción 2

2.1.3. Variables libres y ligadas 2

2.1.4. Sustitución 3

2.1.5. Confluencia 4

2.2. Teoría de tipos 6

2.2.1. Tipado à la Curry y à la Church 13

2.2.2. Lambda cálculo simplemente tipado 13

2.3. Tipos dependientes 14

2.3.1. Tipo Π 16

2.3.2. Tipo Σ 18

2.4. Tipos cociente 19

3. Estado del arte **23**

3.1. Cocientes en teoría de tipos intensional 23

3.2. NuPRL y la teoría de tipos extensional 24

3.3. Tipos cociente en Lean 24

3.4. Formalización de sistemas de tipos en lógica de reescritura 25

4. Maude y la lógica de reescritura **29**

4.1. Sorts y lógica ecuacional ordenada 29

4.2. Módulos funcionales 30

4.3. Módulos de sistema y reescritura no determinista 31

4.4. Formalización de lenguajes y sistemas de tipos 31

5. Un lenguaje dependientemente tipado en Maude **35**

5.1. Decisiones de diseño y alcance 35

5.1.1. Sustitución que evita capturas 35

5.1.2. Anotación explícita de términos 35

5.1.3. Universo único 36

5.1.4. Regla de cómputo del cociente 36

5.2. Sintáxis 36

5.3. Sustitución 37

5.4. Normalización 38

5.5. Verificación de tipos 39

6. Extensión con tipos cociente **43**

6.1. Introducción 44

6.2. Formación 44

6.3. Eliminación y Computación 45

7. Casos de uso y validación **49**

7.1. Los enteros como pares de naturales 49

7.2. Elevación de funciones sobre el cociente 50

7.3. Consistencia sobre representantes 50

7.4. Igualdad proposicional entre representantes 51

7.5. Multiconjuntos bajo relación de permutación 52

8. Conclusiones, limitaciones y trabajo futuro **55**

8.1. Conclusiones 55

8.2. Limitaciones 56

8.3. Trabajo futuro 56

A. Código Maude **61**

B. Contribución a los Objetivos de Desarrollo Sostenible **65**

Capítulo 1

Introducción

1.1. Motivación

La teoría de tipos ocupa hoy un lugar central tanto en los fundamentos de las matemáticas como en el diseño de lenguajes de programación. Desde su origen en la obra de Russell y su posterior desarrollo por Church y Martin-Löf, ha pasado de ser un instrumento para evitar las paradojas de la teoría de conjuntos a convertirse en el sustrato sobre el que se construyen asistentes de pruebas como Agda, Coq y Lean. En estos sistemas, la correspondencia de Curry-Howard permite tratar las proposiciones como tipos y las pruebas como términos que lo habitan, de modo que verificar una demostración se reduce a verificar que un término dado habita el término correspondiente.

Dentro de este marco, los tipos cociente permiten identificar los elementos de un tipo que deberían considerarse iguales según una relación de equivalencia dada. Su utilidad es significativa, dado que aparecen de forma recurrente al formalizar objetos matemáticos como los enteros construidos a partir de pares de naturales, los racionales, o los multiconjuntos. Disponer de cocientes amplía el repertorio de tipos de datos algebraicos y facilita el razonamiento sobre la semántica de programas y explorar conceptos de la teoría de tipos. Ahora bien, los tipos cociente pertenecen a la teoría de tipos extensional, y su incorporación en una teoría intensional, que sostienen a la mayoría de asistentes de pruebas actuales, presenta dificultades ligadas a la decidibilidad de la verificación de tipos y a la preservación de la canonicidad.

La lógica de reescritura, y Maude como su realización práctica, ofrece un ángulo poco explorado para atacar este problema. Maude permite trasladar la sintaxis abstracta de un lenguaje a una signatura, sus reglas de tipado a ecuaciones, y sus identidades estructurales a igualdades ecuacionales, obteniendo así un prototipo ejecutable. El principio de que el cómputo es deducción, propia de la lógica de reescritura, convierte a Maude en un candidato sumamente apropiado para implementar reglas de inferencia y semánticas operacionales. Pese a ello, la formalización de sistemas de tipos dependientes extendidos con tipos cociente sobre lenguajes basados en la reescritura ha recibido escasa atención a lo largo de los años. El presente trabajo se propone cubrir esa brecha, mostrando que Maude constituye una plataforma viable para la formalización, exploración y estudio de la teoría de tipos desde la perspectiva de la reescritura de términos.

1.2. Objetivos

El objetivo general de este trabajo final de máster es diseñar e implementar en Maude un lenguaje de programación dependientemente tipado y extenderlo con tipos cociente. Se busca aprovechar las capacidades de la lógica de reescritura para trasladar de forma natural las reglas de inferencia del sistema de tipos a su forma ecuacional y, en este sentido, interpretar las reglas de inferencia y tipado como una tripla $\mathcal{R} = (\Sigma, E, R)$, de manera que cada juicio de tipado y cada paso de la semántica operacional se corresponda de modo directo con una ecuación del prototipo. Con ello se pretende demostrar que Maude es una herramienta apta para la formalización y el estudio de la teoría de tipos.

1.3. Objetivos específicos

- Diseñar el cálculo lambda dependientemente tipado, especificando en Maude la sintaxis abstracta, el comportamiento de la sustitución y la semántica de normalización ecuacional de un lambda cálculo dependiente, empleando lógica ecuacional.
- Implementar el algoritmo de verificación de tipos desarrollando un verificador de tipos bidireccional mediante ecuaciones condicionales en Maude, garantizando la decidibilidad de la equivalencia de tipos por normalización
- Extender el cálculo dependiente con tipos cociente incorporando las reglas de formación, introducción, eliminación y computación de los tipos cociente intensionales, integrando operacionalmente la reducción del cociente, como una regla de reescritura.
- Validar la consistencia lógica del prototipo evaluando las funciones elevadas ante la elección de representantes equivalentes utilizando la formalización de los números enteros $\mathbb{Z}$ como pares de naturales cocientados y los multiconjuntos bajo la relación de permutación.

1.4. Estructura de la tesis

La memoria se organiza en ocho capítulos. El capítulo 2 presenta los preliminares teóricos, partiendo del cálculo lambda sin tipos y sus propiedades fundamentales, la sustitución que evita capturas, la beta-reducción y la confluencia, para avanzar hacia la teoría de tipos, el cálculo lambda simplemente tipado, los tipos dependientes y un primer acercamiento a los tipos cociente.

El capítulo 3 recorre el estado del conocimiento, revisando el tratamiento de los cocientes en la teoría de tipos intensional, su realización en NuPRL y la teoría extensional, su implementación en Lean mediante tipos cociente basados en setoides, para cerrar con los precedentes de la formalización de sistemas de tipos en lógica de reescritura, partiendo de estudios desarrollados por Blanqui, Stehr y Meseguer.

El capítulo 4 introduce Maude y la lógica de reescritura, describiendo la lógica ecuacional, los módulos funcionales y de sistema, y la metodología general para formalizar lenguajes y sistemas de tipos.

El capítulo 5 desarrolla el núcleo del lenguaje dependientemente tipado, exponiendo las decisiones de diseño y el alcance del prototipo, la sintaxis, la sustitución, la normalización y la verificación de tipos, acompañando cada regla de su realización ecuacional en Maude.

El capítulo 6 aborda la extensión con tipos cociente, detallando las reglas de formación, introducción, eliminación y computación, así como su implementación, y justificando las divergencias respecto de la formulación de Li.

El capítulo 7 valida el prototipo a través de casos de uso, centrándose en los enteros como pares de naturales, en los multiconjuntos bajo la relación de permutación y en la consistencia de las operaciones sobre los distintos representantes de una misma clase.

Finalmente, el capítulo 8 recoge las conclusiones, las limitaciones del trabajo y las líneas de trabajo futuro. En el apéndice A se contempla el código Maude del prototipo.

Capítulo 2

El cálculo lambda y sus extensiones

2.1. El cálculo lambda

El cálculo lambda fue inventado por Alonzo Church en la década de los 30 con el propósito de capturar la noción abstracta de la aplicabilidad de una función, sin embargo, conviene precisar en qué sentido. Una función puede entenderse de dos maneras: de forma extensional, como el conjunto de pares entrada–salida que la determina, o de forma intensional, como la regla o el proceso de cómputo que, dado un argumento, prescribe cómo obtener el resultado (Nederpelt and Geuvers, 2014). Es esta segunda lectura, la de la función como algoritmo, la que el lambda cálculo formaliza (Barendregt, 1984; Nederpelt and Geuvers, 2014). En su origen, Church concibió el sistema como parte de una fundamentación de la lógica y las matemáticas; dicha formulación resultó ser inconsistente, tal y como evidenciaron Kleene y Rosser, por lo que Church aisló su núcleo puramente computacional, libre de constantes lógicas, dando lugar al cálculo lambda tal y como se conoce hoy (Barendregt, 1984), siendo ampliamente adoptado como un sistema formal de lógica matemática.

Este sistema ocupa, además, un lugar central en el diseño de lenguajes de programación. De forma análoga, la manera en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en día. De hecho, lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y Lisp, están ampliamente inspirados en un conjunto del cálculo lambda y pueden entenderse en buena medida como extensiones dotadas de azúcar sintáctica, sistemas de tipos y otras primitivas (Pierce, 2002).

El cálculo lambda es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicación y abstracción lambda. De forma inductiva, fijado un conjunto infinito numerable de variables V, el conjunto Λ de los términos lambda se define como el conjunto de términos bien formados bajo las siguientes reglas (Nederpelt and Geuvers, 2014; Pierce, 2002):

$$\begin{aligned} M := \quad & x && (1)\\ \mid \quad & \lambda x.\, M && (2)\\ \mid \quad & M\ N && (3) \end{aligned}$$

Donde, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$, entonces $x, y, z \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

El lambda cálculo es lo suficientemente potente para permitirnos codificar simples tipos de datos o, al menos, emular el comportamiento que obtendríamos de los mismos en un sistema de tipos más avanzado.

2.1.1. Booleanos

El tipo booleano es un tipo habitado por solo dos posibles términos: $True$ o $False$. Pese a que el cálculo lambda sin tipos no posee una noción de verdad o falsedad, es posible, aún así, codificar los booleanos (Barendregt, 1984):

Ejemplo 2.1. $\text{True} \equiv \lambda xy.\, x, \quad \text{False} \equiv \lambda xy.\, y$

A continuación se detallan propiedades de sumo interés que articulan, posteriormente, los fundamentos del lenguaje que describe esta tésis: la distinción entre variables libres y ligadas, junto con la α-equivalencia que de ella se deriva; la operación de sustitución que evita la captura de variables; y la β-reducción como noción primaria de cómputo. Se mencionará igualmente la η-conversión, que captura la extensionalidad de las funciones.

2.1.2. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\to_\beta$ es empleada. La $\to_\beta$ está basada en la *sustitución*, formalmente expresada como $M[x := N]$, significando que M es un término donde en su cuerpo, la variable x, ha sido reemplazado por N.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como $M[\, x := N]$, donde toda ocurrencia de x en M (estando x libre en M), debe ser reemplazada por N (Nederpelt and Geuvers, 2014).

Ejemplo 2.2. $(\lambda x.\, x + 1)\, 5$ reduce a $(x + 1)[x := 5] = 5 + 1 = 6$.

Ejemplo 2.3. La función identidad aplicada a un término cualquiera devuelve dicho término: $(\lambda x.\, x)\, y \to_\beta x[x := y] = y$.

Ejemplo 2.4. La β-reducción puede encadenarse, reduciendo paso a paso hasta que no queden más redexes. considérese el combinador $K \equiv \lambda x.\, \lambda y.\, x$ aplicado a dos argumentos:

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\, a\, b &\to_\beta (\lambda y.\, x)[x := a]\, b \\ &= (\lambda y.\, a)\, b \\ &\to_\beta a[y := b] = a.\end{aligned}$$

Nótese que el segundo argumento, b, es descartado, pues y no ocurre en el cuerpo del lambda.

2.1.3. Variables libres y ligadas

La abstracción $\lambda x.\, M$ introduce un *binder* sobre la variable x, cuyo *alcance* es el cuerpo de M. Decimos, pues, que una ocurrencia de una variable x está *ligada* cuando aparece dentro del alcance del λx que la introduce; en caso contrario, dicha ocurrencia se dice *libre*.

Conviene resaltar que una misma variable puede ocurrir libre y ligada dentro de un mismo

término. En $(\lambda x.\, x)\, x$, por ejemplo, la primera ocurrencia de x está ligada por el λx, mientras que la segunda, externa a la abstracción, permanece libre. Formalmente, el conjunto de variables libres de un término, denotado $FV(M)$, se define de forma inductiva sobre la estructura del término (Pierce, 2002):

$$
\begin{aligned}
&(1)\ FV(x) = \{x\},\\
&(2)\ FV(\lambda x.\, M) = FV(M) \setminus \{x\},\\
&(3)\ FV(M\, N) = FV(M) \cup FV(N).
\end{aligned}
$$

(1) indica que una variable, por sí sola, es libre. (2) establece que al abstraer sobre x, esta deja de ser libre en $\lambda x.\, M$, pues queda ligada por el nuevo binder; por ello se retira del conjunto. (3) recoge que las variables libres de una aplicación son, simplemente, la unión de las libres de cada subtérmino M y N respectivamente. Un término M tal que $FV(M) = \emptyset$ se denomina *cerrado*.

Por último, el nombre concreto de una variable ligada carece de relevancia, porque lo único que importa es la estructura de binders que establece. Así, $\lambda x.\, x$ y $\lambda y.\, y$ denotan exactamente la misma función identidad. A esta noción, que identifica términos que difieren únicamente en el nombre de sus variables ligadas, se le conoce como α-*equivalencia*, y se denota $=_\alpha$. El proceso de renombrar una variable ligada para obtener un término α-equivalente recibe el nombre de α-*conversión*.

2.1.4. Sustitución

Anteriormente, definimos la sustitución en términos de $\to_\beta$ reducción, sin embargo, la sustitución posee en sí reglas de cómputo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto alguno.

Se definen, pues, las reglas de cómputo de la sustitución de acuerdo a lo discutido por Pierce (2002). Formalmente:

$$
\begin{aligned}
&(1)\ x[x := N] \equiv N,\\
&(2)\ y[x := N] \equiv y \text{ if } x \not\equiv y,\\
&(3)\ (PQ)[x := N] \equiv (P[x := N])(Q[x := N]),\\
&(4)\ (\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y\to z}[x := N]),\ \text{si } \lambda z.\, P^{y\to z} \text{ es un } \alpha\text{-variante de}\\
&\quad \lambda y.\, P \text{ tal que } z \notin FV(N).
\end{aligned}
$$

Para (1), sustituyendo x, por N dentro de x, siendo x el término que buscamos sustituir, obtenemos N. De forma análoga, sustituyendo cada ocurrencia de x en el término y, por N, si x e y no son iguales, el término intacto, y, será retornado. El caso de (3), tiene sentido cuando P es una abstracción aplicada a un término Q. Concretamente, se reduce a la sustitución de x por N, en ambos P y Q. (4) contempla la sustitución bajo un binder. Lo que se busca es sustituir x por N dentro del cuerpo P de la abstracción $\lambda y.\, P$. Sin embargo, si procediéramos de forma ingenua y N contuviera a y como variable libre, dicha y quedaría equivocadamente atrapada por el binder λy, alterando por completo el significado del término. A este fenómeno se le conoce como *captura de variables*.

Para evitarlo, antes de sustituir se renombra la variable ligada y por una variable fresca z, mediante α-conversión, escogida de modo que $z \notin FV(N)$ (y, por extensión, que no genere conflicto con las variables ya presentes en P). Solo entonces se procede a sustituir x por N dentro del cuerpo ya renombrado. De esta manera, la sustitución preserva el significado del término original, razón por la cual se la describe como una sustitución *que evita la captura* (Pierce, 2002).

Ejemplo 2.5. Supóngase la sustitución $(\lambda y.\, x)[x := y]$. Si sustituyéramos directamente, obtendríamos $\lambda y.\, y$, es decir, la identidad: la y libre que introdujimos habría sido capturada por el binder. Renombrando primero la variable ligada a una fresca z, obtenemos $(\lambda z.\, x)[x := y] = \lambda z.\, y$, que es el resultado correcto, pues la y permanece libre.

2.1.5. Confluencia

Anteriormente señalamos cómo la β-reducción emula, en cierto sentido, el proceso de cómputo de una función dada: evaluado un término, aspiramos a obtener su forma normal, esto es, un término en el que ya no resta redex alguno que nos permita seguir reduciendo (Barendregt, 1984; Pierce, 2002).

Ahora bien, el orden en que se contraen tales redexes puede influir en si el cómputo termina o, mejor dicho, en si alcanza una forma normal. Surge así una pregunta: dado un término t del que divergen dos reducciones, $t \to^*_\beta t_1$ y $t \to^*_\beta t_2$, en uno o varios pasos, ¿es posible reconciliar ambas ramas en un término común u alcanzable tanto desde t_1 como desde t_2? Cuando tal u existe para cualquier par de reducciones divergentes, decimos que la reducción es *confluente* sobre t.

En el presente apartado describimos con detenimiento la propiedad de confluencia y, exponémos también, por qué la β-reducción la satisface.

La confluencia, conocida también como teorema de Church-Rosser, es una propiedad de los sistemas de reescritura que caracteriza cuándo dos términos, derivados de un término inicial, por reducción en uno o más pasos, convergen a un mismo resultado. Su relevancia radica en que el resultado del cómputo queda determinado con independencia del orden en que se ejecuten los pasos de reducción (Barendregt, 1984).

Teorema 2.1 (Church-Rosser)**.** *Sea M un término del λ-cálculo. Si $M \to^*_\beta N_1$ y $M \to^*_\beta N_2$, entonces existe un término N_3 tal que $N_1 \to^*_\beta N_3$ y $N_2 \to^*_\beta N_3$ (Barendregt, 1984, p. 23).*

Conviene precisar que el teorema habla de la *clausura reflexivo-transitiva* de la reducción, $\to^*_\beta$, es decir, de cero o más pasos de β-reducción. Es importante resaltar que la β-reducción no cumple la versión más fuerte de la propiedad, conocida como propiedad del diamante en un solo paso: si de M contraemos dos redexes distintos en un único paso para obtener N_1 y N_2, no siempre existe un N_3 alcanzable desde ambos en un solo paso. La razón es la duplicación, pues un redex puede copiar a otro cuando el argumento contiene, a su vez, redexes, de modo que un lado de la bifurcación necesita varios pasos para reabsorber las copias que el otro lado ya había contraído. Lo que el teorema de Church-Rosser garantiza es la versión global, sobre $\to^*_\beta$, que sí se sostiene.

Ademas, por simplicidad, hasta ahora hemos de forma ligera abusado del término de confluencia. Resulta importante separar dos nociones de la misma: la confluencia local y la confluencia

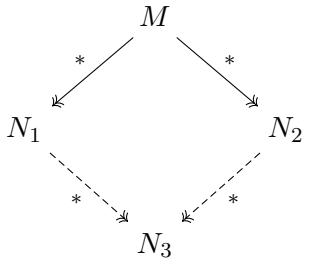


Figura 2.1: Confluencia

global. Decimos que un sistema es *localmente confluente* cuando, partiendo de M y dando un paso hacia N_1 y otro hacia N_2, existe un término común alcanzable desde ambos en cero o máas pasos. La confluencia local es más débil que la confluencia global, y la implicación de la primera a la segunda no siempre se cumple, pues solo se obtiene bajo hipótesis adicionales, como la terminación, por tanto, una relación terminante es confluente, si es localmente confluente. de acuerdo al lema de Newman. (Baader and Nipkow, 1998; Barendregt, 1984).

Lema 2.2. *M es una forma normal si M no contiene más redexes. Luego, M tiene una forma normal si existe un N, en forma normal, tal que $M =_\beta N$. Tal N es la forma normal de M.*

Una consecuencia inmediata, y quizá la más útil en la práctica, es la unicidad de la forma normal. Si un término admite forma normal, esta es única salvo α-equivalencia.

Corolario 2.3. *Si M posee una forma normal, esta es única salvo α-equivalencia.*

En efecto, supóngase que N_1 y N_2 son ambas formas normales de M, de modo que $M \to_\beta^* N_1$ y $M \to_\beta^* N_2$. Por el teorema de Church-Rosser existe N_3 con $N_1 \to_\beta^* N_3$ y $N_2 \to_\beta^* N_3$. Pero N_1 y N_2 están en forma normal, luego no contienen redex alguno y no pueden reducirse en pasos no triviales, de donde $N_1 =_\alpha N_3 =_\alpha N_2$.

Este corolario, por un lado, justifica que hablemos de *la* forma normal de un término, en singular, cuando esta existe. Por otro, garantiza que el valor de un cómputo, entendido como su forma normal, queda determinado por el término de partida y no por la estrategia de evaluación escogida, de modo que el orden en que decidamos contraer los redexes solo puede afectar a si terminamos, mas no al resultado al que llegamos.

La confluencia se representa mediante la siguiente figura 2.1, cuya forma de rombo explica que la propiedad reciba también el nombre de *propiedad del diamante*. El vértice superior es el término de partida M, las dos diagonales descendentes son las reducciones divergentes hacia los términos N_1 y N_2, y las diagonales punteadas representan los pasos que aún restan, garantizados por el teorema, para reconciliar ambas ramas en un término común N_3.

Ejemplo 2.6. Considérese el término $M \equiv (\lambda x.\, x\, x)\, ((\lambda y.\, y)\, z)$, que presenta dos redexes: el externo, que aplica $\lambda x.\, x\, x$ a su argumento, y el interno, $(\lambda y.\, y)\, z$. Contrayendo primero el redex externo, el argumento se duplica y obtenemos $N_1 \equiv ((\lambda y.\, y)\, z)\, ((\lambda y.\, y)\, z)$, que aún requiere dos pasos para resolver las dos copias del redex interno. Contrayendo primero el redex interno, en cambio, llegamos en un solo paso a $N_2 \equiv (\lambda x.\, x\, x)\, z$. Ambas ramas convergen en $z\, z$:

Obsérvese cómo la rama izquierda necesita más de un paso para alcanzar la confluencia, lo cual ilustra por qué el teorema se enuncia sobre $\to_\beta^*$ y no sobre la reducción en un único paso.

De aquí emerge el problema de la terminación. Esta determina si el cómputo de un término concluye o no. En el λ-cálculo, una secuencia de reducciones puede no detenerse, entrando en un ciclo infinito de pasos que impide alcanzar forma normal alguna, como ilustra el siguiente ejemplo.

Ejemplo 2.7. El ejemplo canónico de término sin forma normal es el combinador $\Omega \equiv (\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$. Al contraer su único redex, el argumento $\lambda x.\, x\, x$ se sustituye en el cuerpo $x\, x$, devolviendo de nuevo Ω:

$$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \to_\beta (x\, x)[x := \lambda x.\, x\, x] = (\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \equiv \Omega.$$

La reducción se cierra pues, sobre sí misma:

$$\Omega \equiv (\lambda x.\, x\, x)\, \overbrace{(\lambda x.\, x\, x)}^{\beta}$$

El término es trivialmente confluente, ya que su única reducción conduce siempre a sí mismo, y sin embargo carece de forma normal. Este ejemplo muestra que la confluencia y la terminación son propiedades independientes, o en otras palabras, un sistema puede ser confluente sin ser necesariamente terminante.

El ejemplo de Ω pone de manifiesto que el λ-cálculo sin tipos no es terminante, pues efectivamente existen términos que carecen de una forma normal y cuya reducción no se detiene en ningún momento. La terminación, entendida como normalización fuerte, se recupera al imponer que cada término esté acompañado del tipo al que pertenece. Particularmente, el λ-cálculo simplemente tipado sí es fuertemente normalizante, propiedad que abordarémos en un capítulo posterior, al tratar dicho sistema.(Nederpelt and Geuvers, 2014; Pierce, 2002).

2.2. Teoría de tipos

La teoria de tipos es una rama de las matemáticas y la lógica que tiene como objeto formalizar no solo términos matemáticos, tales como una variable x o una función cualquiera f y las operaciones entre ambas, si no también, la idea de que cada término forma parte o tiene un tipo concreto, es decir, que el tipo de una variable $x : \mathbb{N}$ en el dominio de los numeros naturales, es completamente distinguible al tipo de una función $f : \mathbb{N} \to \mathbb{N}$.

Los tipos, pese a que son similares a los conjuntos, son ligeramente distintos. Por conjunto, entiéndase como una colección de elementos, similares o no, que poseen una propiedad concreta:

$$C = \{rojo, amarillo, azul\} \tag{2.1}$$

de donde es posible extenderlos y razonar sobre más relaciones de pertenencia y formar, por ejemplo, conjuntos dentro de otros conjuntos y definir que un elemento pertenece a varios conjuntos a la vez:

$$C' = \{\text{verde}, \text{naranja}\} \cup C \tag{2.2}$$

Cosa que, en los tipos, no es posible, y es que un término es de (o pertenece a) un tipo único. Otra característica que los distingue, es que los conjuntos dan información semántica y, los tipos

por otro lado, proveen información sintáctica de los objetos. Por ejemplo, comunmente en la teoría de conjuntos definimos una relación de pertenencia como sigue:

$$4 \in \{\, n \in \mathbb{N} | x > 5 \,\} \tag{2.3}$$

En la teoría de tipos, un juicio de tipado, sigue la siguiente notación que, a lo largo de la presente tesis estarémos empleando:

$$3 + 1 : \mathbb{N} \tag{2.4}$$

Esto puede interpretarse, desde el lente de teoría de tipos, como $3 + 1$ es un término en los naturales, o bien, de tipo $\mathbb{N}$. Aún así, las mismas operaciones aplicadas en la teoría de conjuntos son trasladables a la teoría de tipos. Tómese como ejemplo la unión disjunta de conjuntos, definida formalmente como la unión disjunta $A \sqcup B$ de dos conjuntos, es un conjunto A indexado por B:

$$\bigcup_{i \in I} \{(x, i) : x \in A_i\} \tag{2.5}$$

Esta operacion resulta en el tipo de dato algebraico $\tau_1 + \tau_2$, donde el valor concreto del tipo puede bien ser la proyección del lado izquierdo o el derecho.

La noción de tipos se atribuye a Bertrand Russell, quien desarrolló la teoría de tipos en respuesta a las contradicciones que, a comienzos del siglo XX, aquejaban a los fundamentos de la matemática. El contexto es el del programa logicista y la teoría de conjuntos naciente. Hacia finales del siglo XIX, Georg Cantor había desarrollado la teoría de conjuntos como un marco general en el que toda la matemática podía, en principio, expresarse, y Gottlob Frege perseguía un objetivo todavía más ambicioso en sus *Grundgesetze der Arithmetik*: reducir la aritmética a la lógica pura. Ambos proyectos descansaban, de forma más o menos explícita, sobre el principio de comprensión irrestricta, según el cual toda propiedad bien definida determina un conjunto, el de todos los objetos que la satisfacen (Coquand, 2022).

Russell advirtió en 1901 que ese principio conducía a una contradicción. Si toda propiedad define un conjunto, entonces la propiedad de no pertenecer a sí mismo define el conjunto de todos los conjuntos que no son miembros de sí mismos:

$$R = \{\, x \mid x \notin x \,\} \tag{2.6}$$

Preguntarse si R pertenece o no a sí mismo lleva, por tanto, a una contradicción lógica, pues por la propia definición de R se tiene que:

$$R \in R \iff R \notin R \tag{2.7}$$

es decir, R es miembro de sí mismo exactamente cuando no lo es.

Lo que Russell sugeria pues, es que estas paradojas comparten una misma estructura, todas involucran una definición que se refiere a una totalidad de la que el objeto definido forma parte. A esto lo llamó el principio del círculo vicioso, que prohíbe que una entidad se defina en términos de una colección a la que ella misma pertenece. La expresión $x \in x$, sobre la que se construye R, es precisamente una de estas definiciones circulares, o autoreferencia.

Para evitarlas, Russell propuso organizar los objetos en una jerarquía estratificada. En el nivel más bajo están los individuos; en el siguiente, los predicados (o clases) de individuos; después los predicados de predicados, y así sucesivamente. Esta distinción es la que la teoría de tipos internaliza: a cada objeto se le asigna un tipo, y una función proposicional solo puede aplicarse a argumentos del tipo inmediatamente inferior. Bajo esta restricción, la fórmula $x \in x$ ni siquiera llega a estar bien y es rechazada, ya que la pertenencia exige que su argumento sea de un tipo menor que el del conjunto. El conjunto R no puede construirse, y la paradoja desaparece por construcción. De este modo, conjuntos grandes y pequeños residen en niveles distintos de la teoría.

En términos más generales, los tipos son ubicuos en computación, y numerosos lenguajes tienen su propio sistema de tipos, unos mas avanzados que otros y, otros más simples. Tan simples que la manipulación de tipos es casi perceptible o inexitstente. Lenguajes con esta capacidad de omitir tipos son conocidos como lenguajes dinámicamente tipados, donde en casos, es completamente opcional tipar expresiones y en caso contrario, el compilador hace lo posible para inferir un tipo adecuado para una expresión. Haskell por ejemplo, es un lenguaje de alto nivel, ampliamente utilizado por grupos de investigación y académicos, su sistema de tipos esta inicialmente basado en el cálculo lambda simplemente tipado, también llamado *System F*, que abordaremos en capítulos posteriores a este. Sin embargo, hoy en día su sistema de tipos ha madurado lo suficiente como para dejar de considerarle un lenguaje con un sistema de tipos simple. C++ es un claro ejemplo de un sistema de tipos, quizá, mucho más complejo, pero sumamente rico, que facilita posteriormente, una distincion semántica y rechaza programas que son imposibles de evaluar correctamente. El lenguaje en el que estarémos trabajando en el presente trabajo, Maude, es de alto nivel pero fuertemente tipado. Aunque precisamente no utiliza tipos internamente, sino Sorts, la manipulación de términos y el enunciar si un término es de tipo (o sort) A es, en parte, lo mismo. OCaml en principio, fue de los lenguajes que llego a explorar más a fondo la teoria de tipos y la expresividad de los mismos como funcionalidades del lenguaje, siendo de los primeros en implementar tipos polimórficos, es decir, tipos que pueden tomar la forma de otros tipos. Finalmente, un sistema de tipos permite al lenguaje alcanzar capacidades de expresividad bastante amplias y, a su vez, evitar que programas erronoes se ejecuten.

Dentro de los problemas que intenta resolver la teoria de tipos, no se encuentra solo encontrar un tipo para un término concreto, sino también, determinar (si lo hay) un término para un tipo y si un término está bien tipado (o es un tipo legal) dentro de un contexto (Nederpelt and Geuvers, 2014). Listamos pues, algunos de los problemas que se abordan en la teoria de tipos:

1) Tipabilidad: este problema formula la pregunta: ¿existe un tipo bien tipado para el término en concreto en el contexto?

$$\Gamma \vdash a :?$$

No siempre existe un tipo bien definido para un término. Las derivaciones o juicios de tipado sirven justamente para determinar logicamente cuál tipo es el adecuado o, correctamente, inferir el tipo del término. No encontrar dicho tipo, por supuesto, ocasionaría un error de tipado.

2) Type-checking: determina si el tipo de un término en un contexto Γ cualquiera, es correcto o bien tipado.

$$\Gamma \overset{?}{\vdash} a : A$$

Sea $\Gamma = \{b : B, U : *\}$, si A no es un tipo bien formado en Γ, el sistema de tipos lo rechaza. Esto es claro, pues, no hay una regla que permita derivar que $a : A$ está bien formado en Γ.

3) Habitabilidad: el problema de la habilitabilidad, también llamado *term finding*, formula la pregunta: ¿existe un término, dado un tipo bien definido y un contexto concreto?

$$\Gamma \overset{?}{\vdash} ? : A$$

Nótese que los problemas anteriormente mencionados son decibibles en $\lambda \rightarrow$, es decir, para cada una de estas cuestiones existe un algoritmo posible que produce un resultado en el lambda cálculo simplemente tipado. En sistemas de tipos más complicados, sin embargo, la búsqueda de un término llega a ser un problema, pues tales sistemas no logran encontrar un habitante del tipo en el contexto, siendo problemas no decidibles (Barendregt et al., 2013).

Se empleará, a lo largo de la tesis, la siguiente notación para definir las derivaciones en la teoría de tipos, donde la parte superior es una lista de premisas y la parte inferior, la conclusión o los consecuentes:

$$\frac{premisa_1 ... \quad premisa_n}{consecuente_1 ... consecuente_n} \tag{2.8}$$

Para $n \geq 0$. Las reglas pueden leerse bien de arriba hacia abajo (como *teniendo que, podemos concluir que*) o de abajo hacia arriba (como *dado que se cumple el consecuente, las premisas deben cumplirse*). Adicionalmente, las variables presentes en premisas y consecuentes, estan universalmente quantificadas a menos de que se indique lo contrario. Por tanto,

$$\frac{}{\forall x, P(x)} \tag{2.9}$$

Donde $\forall$ es implícito.

$$\frac{\Gamma \vdash A \quad \Gamma, a : A}{\Gamma \vdash a : A} \tag{2.10}$$

Que puede leerse como: *si, dado que en el contexto* Γ*,* A *es un tipo bien formado y, además, en el mismo contexto* Γ *extendido con* $a : A$*, entonces podemos inferir que* a *es de tipo* A *en* Γ.

Dentro de la teoría de tipos existen seis tipos de juicios que serán de utilidad en la medida en que se estudiarán los sistemas formales en este trabajo:

$\vdash \Gamma\ ctx$	Γ es un contexto valido
$\Gamma \vdash \sigma\ Type$	σ es un tipo
$\Gamma \vdash M : \sigma$	M es un término de tipo σ en Γ
$\vdash \Gamma \equiv \Delta$	$\Gamma\ y\ \Delta$ son contextos definicionalmente iguales
$\Gamma \vdash \sigma \equiv \tau\ Type$	$\sigma\ y\ \tau$ son tipos definicionalmente iguales en Γ
$\Gamma \vdash M \equiv N : \sigma$	$M\ y\ N$ son términos definicionalmente iguales, de tipo σ, en Γ

La primera regla, hace referencia a la formación de contextos. La misma simplemente dice que *existe* un contexto, sin ningunas suposiciones previas:

$$\frac{}{\vdash \Gamma ctx} \tag{2.11}$$

Las reglas que le siguen, hacen alusión a la extensión de contextos y variables. Con ello, es posible construir el siguiente árbol de derivaciones, como sigue:

$$\frac{\dfrac{\Gamma\ \mathsf{ctxt} \qquad \Gamma \vdash A : \mathsf{Type}}{\Gamma, x : A\ \mathsf{ctxt}}}{\Gamma, x : A \vdash x : A} \tag{2.12}$$

Así, dado el juicio resultante donde se cumple que $x : A$ en Γ, primero ha debido de ser cierto que $x : A$ es un término bien tipado en el contexto ctx y, anterior a ello, que Γ es un contexto bien definido y A habita $Type$ en el mismo. Nótese que la regla asume que no hay otra variable x en Γ, en otras palabras, que x es una variable fresca.

Adicionalmente, independientemente del sistema de tipos que se describa, las mismas reglas estructurales se adoptan, como bien describe Hofmann, 2009.

$$\frac{}{\vdash \emptyset}\ \text{C-Emp} \qquad \frac{\Gamma \vdash \sigma}{\vdash, \Gamma\ x : \sigma}\ \text{C-Ext} \tag{2.13}$$

$$\frac{\vdash \Gamma, x : \sigma}{\Gamma \vdash x : \sigma}\ \text{Var} \tag{2.14}$$

Sobre estos juicios, la igualdad definicional se comporta como una relación de equivalencia, esto es, es reflexiva, simétrica y transitiva, tanto para tipos como para términos, y es además una congruencia respecto de los constructores del lenguaje. A ella se añade la regla de conversión, que permite tratar un término de tipo σ como un término de tipo τ siempre que $\sigma \equiv \tau$. No se detallan aquí el resto de reglas correspondientes, sin embargo, su formulación completa puede consultarse en (Hofmann, 2009). En el prototipo que se describe en el capítulo 5, esta igualdad se decide por normalización de ambos lados y comparación de sus formas normales, según se detalla en la ecuación `tyEq`, en lugar de axiomatizarse mediante tales reglas.

A lo largo de los años numerosos esfuerzos se han realizado para desarrollar distintas extensiones de la teoría de tipos y poder dar suficiente poder expresivo al lambda cálculo. Estos sistemas de tipos fueron explorados por H. Barendregt en el Cubo Lambda (Barendregt, 1991), para investigar las diferentes dimensiones en las que puede extenderse el sistema formal. En

su paper describe como cada dimensión del cubo representa una dependencia entre términos y tipos, y sus posibles combinaciones.

Entiéndase por dependencia como la dependencia mutua, valga la redundancia, que existe entre tipos y términos: términos que dependen de otros términos, términos dependientes de tipos, tipos dependientes de términos, y tipos dependientes de otros tipos. Los ocho vertices del cubo, consecuencia de combinar cada posible dimensión, resultan en diferentes sistemas de tipos y niveles de expresividad. Concretamente, abordaremos en el presente trabajo, la extension del vertice inferior derecho, que corresponde a la extension de tipos dependientes, concreamente $\lambda\Pi$, o refiérase también en la literatura como λP.

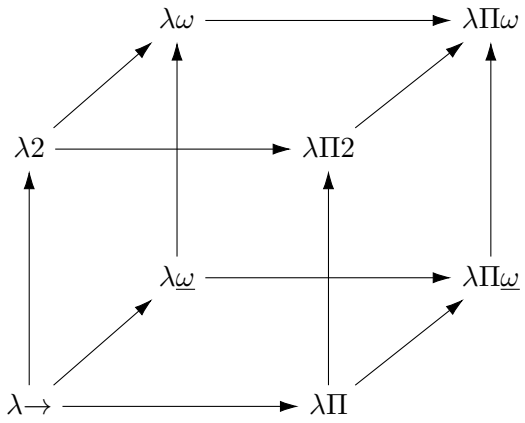


Figura 2.2: El cubo lambda de Barendregt

La dependencia que define cada vertice, es descrita por la siguiente tabla:

Sistema	Conjunto de reglas			
$\lambda\!\rightarrow$	$(\star,\star)$			
$\lambda 2$	$(\star,\star)$	$(\Box,\star)$		
λP	$(\star,\star)$		$(\star,\Box)$	
λP2	$(\star,\star)$	$(\Box,\star)$	$(\star,\Box)$	
$\lambda\underline{\omega}$	$(\star,\star)$			$(\Box,\Box)$
$\lambda\omega$	$(\star,\star)$	$(\Box,\star)$		$(\Box,\Box)$
$\lambda\mathrm{P}\underline{\omega}$	$(\star,\star)$		$(\star,\Box)$	$(\Box,\Box)$
$\lambda\mathrm{P}\omega = \lambda$C	$(\star,\star)$	$(\Box,\star)$	$(\star,\Box)$	$(\Box,\Box)$

Es necesaria una explicación a los conjuntos de reglas más relevantes y nivel de expresividad que permite cada sistema. $\lambda\!\rightarrow$ es posiblemente el más común. Formalmente este sistema es conocido como el lambda cálculo simplemente tipado que se abordará con más dedicación en un capítulo aparte, donde para construir una abstracción lambda, basta con proveer otro término. Por tanto FM es, por ejemplo, un término en Λ donde F depende de M. La notación $(\star,\star)$ indica precisamente en qué nivel del sistema de tipos viven los términos para poder formarse.

Ejemplo 2.8.

$$(\lambda x : \mathbb{N}.t)3 \quad \longrightarrow t[x := 3] \longrightarrow 3 \tag{2.15}$$

$\lambda 2$ es conocido en la literatura como *System F*. Es el lambda cálculo de segundo orden, o polimórfico. Fue descubierto por Girard y Reynolds en 1972 en el cual se introduce el cuantificador universal en el cálculo y poder cuantificar sobre tipos. A parte de ello, a partir de aquí algunas implementaciones extienden el sistema introduciendo kinds en la teoría impredicativa del lenguaje. Un gran número de lenguajes de programación funcional, como Haskell, están inspirados en este sistema. Además, en $\lambda 2$ se presenta la noción de variables de tipos, con las cuales es posible cuantificar sobre un rango de tipos, permitiendo así, la suficiente expresividad al lambda cálculo para definir, por instancia, la función identidad polimórfica.

Ejemplo 2.9.

$$(\Lambda\alpha\lambda x : \alpha.x)\, 3 \longrightarrow_\beta 3$$
$$(\Lambda\alpha\lambda x : \alpha.x)\, \mathit{true} \longrightarrow_\beta \mathit{true}$$

Donde Λ representa el cuantificador universal sobre los tipos del lenguaje. De forma interna, un tipo es aplicado al esquema del tipo mediante instanciación, capturando la noción de polimorfismo.

$\lambda\underline{\omega}$ añade operadores a nivel de tipos, o bien, funciones que toman uno o varios tipos y devuelven otro. Para ello se introduce un sistema de *kinds* que distingue los tipos propios, de kind $\star$, de los constructores de tipos, de kind $\star \Rightarrow \star$. Un tipo puede entonces depender de otros mediante abstracción y aplicación a nivel de tipos, del mismo modo que un término depende de otros a nivel de valores.

Ejemplo 2.10. El constructor de pares toma dos tipos y devuelve el tipo de la tupla.

$$\mathit{Pair} \;\equiv\; \lambda\alpha : \star.\; \lambda\beta : \star.\; \alpha \times \beta$$

Así, *Pair* Nat Bool reduce a Nat $\times$ Bool.

$\lambda\omega$ combina los operadores de tipos anteriormente presentados y el polimorfismo de $\lambda 2$, así, ahora es posible cuantificar universalmente sobre tipos y constructores de tipos.

Ejemplo 2.11. Como ejemplo tómese la función *swap* que intercambia la posición de los elementos en una tupla. Cuantificando ambos argumentos α y β, podemos aplicar *swap* sobre cualquier tipo que habite $\star$.

$$\mathit{swap} \;:\; \forall\alpha : \star.\; \forall\beta : \star.\; \mathit{Pair}\ \alpha\ \beta \to \mathit{Pair}\ \beta\ \alpha$$

λP representa los tipos que dependen de términos. Los tipos dependientes, como se les referirá a lo largo de este trabajo, extienden el sistema de tipos con familias de tipos, o tipos indexados por los elementos de otros tipos. La siguiente regla de inferencia formaliza esta idea donde x puede o no, aparecer en B. Obsérvese que de no aparecer x en B, estaríamos involucrándonos con el tipo de función $A \to B$.

$$\frac{\Gamma \vdash A : \star,\ x : A \quad \Gamma \vdash B : A}{\Gamma \vdash \Pi x : A.Bx} \tag{2.16}$$

Tómese como ejemplo *Vec* A n donde la longitud n del vector, depende de cierto parámetro. Decimos, por tanto, que *Vec* A n es un tipo dependiente o bien una familia de tipos

indexado por n. Asimismo, la linea con la que se distinguen términos y tipos se hace más difusa y, en consecuencia, ambos comparten el mismo espacio semántico. Por ello, encontramos la constante $\star$ que representa el *sort* en el que habitan todos los tipos, por tanto, la siguiente derivación es posible en λP: $A : \star \vdash (A \to A) : \star$, que expresa que *dado que A es un tipo bien formado, $(A \to A)$, es entonces, un tipo* (Barendregt, 1991). El símbolo $\Box$ no es propiamente un término en la sintaxis, en cambio, es un símbolo en la meta-teoría para afirmar que se tiene una derivación de kinds válida y predicativa, por tanto, se puede entender $\Gamma \vdash (\kappa : \Box)$ mediante la siguiente lectura: *k es un kind bien formado en el contexto* Γ.

Combinando todos los vértices obtenémos λC o *Calculus of Constructions* donde el lambda cálculo alcanza un nivel expresivo bastante avanzado y los sistemas del cubo cohabitan en uno solo. Lenguajes como Coq y Agda, están basados en λC. Este es un sistema de tipos puro bajo el paradigma de proposiciones como tipos, con tipos dependientes, jerarquias de universos de tipos y un tipo Prop que distingue de $\star$.

2.2.1. Tipado à la Curry y à la Church

En el lenguaje que se formaliza en el presente trabajo, cada variable lleva consigo su tipo. Pese a que garantiza decibilidad en cada expresión que se construya, esta decisión no es arbitraria. Describe Reynolds (2000) que existen dos formas en las que es posible entender qué significa que una expresión tenga un tipo. En la semántica intrínseca solo los términos, a los que Reynolds se refiere como frases, guardan un sentido si satisfacen un juicio de tipado y, por otro lado, una expresion carece de sentido si no satisface ningun juicio, pero de satisfacer varios, poseerá tantos significados como juicios.

El tipado *à la Church*, que es la forma que adoptarémos, deja en evidencia de forma explícita el tipo al que pertenece una variable ligada, por otro lado, el tipado à la Curry omite las anotaciones y el chequeo para determinar su tipo es delegado a otro proceso de búsqueda o inferencia. La ventaja que se manifiesta en el tipado à la Church es que la verificación del tipo en Maude viene dada por pattern matching y, de manera ecuacional, restringimos que cada término bien formado admita un único tipo o pertenezca a un tipo en concreto.

Considérese como ejemplo la expresión $\lambda x : int.x$ su significado intrínseco es la función identidad sobre los enteros. La identidad sobre los booleanos, por otro lado, que se denota como $\lambda x : bool.x$ es una función distinta y, por tanto, un término distinto. Es decir, el tipo determina el significado del término. Por el contrario $\lambda x.x$, su significado extrínseco es la función identidad sobre el universo de valores que pudiesen aparecer en un cómputo, por tanto, los tipos $bool \to bool$, $int \to int$ se le atribuyen de forma externa, como predicados que dicho significado satisface.

2.2.2. Lambda cálculo simplemente tipado

Añadir tipos al lambda cálculo resuelve el problema de la terminación que en apartado 2.1.5 estudiamos, permitiendo la creación y evaluación de términos en el cálculo mucho más restrictivo, esto es, anotando cada término con un tipo previamente definido que nos brindará un poco más de información sobre el dominio o tipo de valor que esperamos retornar. Este sistema es también denotado como $\lambda \to$ y en definitiva, tiene como objeto evitar anomalías que permitan la construcción de programas no terminantes o no lógicamente plausibles (Nederpelt and Geuvers, 2014).

La forma canónica de implementar tipos en λ es, en principio, definir un conjunto infinito de variables $\mathbb{V} = \{\alpha, \beta, \gamma\}$ y un conjunto $\mathbb{T}$ de tipos: $\tau, \tau \to \tau$. Los miembros de este conjunto estan definidos como:

1. Variable de tipo: donde si $\alpha \in \mathbb{V}$ entonces $\alpha \in \mathbb{T}$
2. Tipo función: donde si $\alpha, \tau \in \mathbb{T}$ entonces $(\alpha \to \tau) \in \mathbb{T}$

La gramática para el lambda cálculo simplemente tipado, se extiende, como se muestra:

$$M = \ldots \mid x : \tau \mid \lambda x : \tau . e$$
$$\mathbb{T} = \{\tau \mid \tau \to \tau\}$$

Los tipos en $\lambda \to$ nos permiten enunciar juicios de la forma $M : \tau$, que se entiende como *el término M es de tipo τ*, asumiendo que $M \in \Lambda$ y τ es un tipo bien definido. A continuación, se explican algunos detalles a tomar en cuenta para cada término que, desde ahora, llevan consigo un tipo.

1. Aplicación: para cubrir el caso donde $M \cdot N$, es menester conocer la signatura de M. Sea M y N términos en Λ, donde M es de tipo $\sigma \to \tau$ y N de tipo σ, es claro observar que N posee el mismo tipo que el del dominio de M por tanto, es acertado confirmar que el tipo de la aplicación es de tipo $M \cdot N : \tau$.
2. Abstracción: cuando $\lambda x : \tau . M$ donde $M : \gamma$, podemos confirmar que es una abstraccion con codominio en γ y dominio en τ.

2.3. Tipos dependientes

Los tipos dependientes son, en principio, tipos que dependen de términos. El ejemplo canónico es el vector el cual su longitud depende de un valor n de tipo $\mathbb{N}$. Puesto a que un tipo arbitrario puede aparecer dentro de un tipo, la igualdad entre tipos requiere determinar si dos programas producen (reducen a) el mismo resultado, por tanto, la igualdad empieza a no ser decidible. Los tipos dependientes conforman parte del trabajo de Per Martin-Löf sobre la teoría de tipos intuicionista, que tiene como objeto en gran parte extender la interpretación de proposiciones como tipos en la lógica de predicados (Martin-Löf, 1975), como estarémos estudiando en este capítulo. Antes de adentrarnos en la teoría de tipos dependientes, es necesario dedicar de forma breve una introducción a los sistemas de tipos puros, framework de donde parte esta formalización lógica. Un PTS es un framework para especificar y formalizar el lambda cálculo y extender su teoría. Fue propuesto inicialmente por Berardi y Terlow como una generalizacion del cubo lambda de Barendregt (1991). A diferencia del lambda cálculo tradicional, un PTS dispone de pseudo-términos que heredan, en gran parte, sintáxis del λ cálculo simplemente tipado que hasta ahora conocemos.

El conjunto de pseudo-términos, se define como sigue:

Definición 2.1. Pseudo-términos de un PTS

$$\mathcal{T} = \mathcal{T} \mid \mathcal{T}\,\mathcal{T} \mid \mathcal{S} \mid \lambda \mathcal{V} : \mathcal{T} . \mathcal{T} \mid \Pi \mathcal{V} : \mathcal{T} . \mathcal{T}$$

Asimismo, resaltan Barthe and Coquand (2002) algunas características a tomar en cuenta que ofrecen una imágen más amplia de la definición de un sistema de tipos puro.

1. Se utiliza la misma convención para extraer el conjunto de variables libres en un pseudo-término $FV(M)$. Adicional a ello, se escribirá $A \to B$ para todo tipo de función en el que x no aparezca libre en B. Y se usará $\Pi x : A.B(x)$, en caso contrario.

2. Los sistemas de tipos puros, describen un λ cálculo *à la Church*, es decir, las abstracciones lambda llevan consigo el dominio de las variables ligadas y, como consecuencia de ello, la computabilidad en un sistema de tipos puro es decidible.

3. Los sistemas de tipos puros proveen los cimientos de los tipos dependientes, de hecho, es en el constructor del tipo Π donde se evidencia, por primera vez, la dependiencia en esta teoría, capturando la noción del producto cartesiano generalizado.

La noción de tipo dependiente es análoga al producto cartesiano dependiente, o generalizado de la teoría de conjuntos. En los conjuntos es posible, para cada conjunto A, y una familia de conjuntos $B(x)_{x \in A}$ indexados por A, formar un nuevo conjunto $\Pi_{x \in A} B_x$, donde sus elementos son funciones con dominio en A, tal que $f(a) \in B_a$ para todo $a \in A$. Consideramos, entonces, $\Pi_{x \in A} B_x$ un producto cartesiano dependiente, pues B_a en el cual $f(a)$ vive, depende de a. De igual forma opera el tipo Π que hemos mencionado en el conjunto de pseudo-términos que conforman un PTS: sea A un tipo definido en Γ y $B(x)$ un tipo, donde x es una variale de tipo A, decimos pues, que $\Pi x : A.B(x)$ es un tipo dependiente donde para todo x en A, existe una familia de tipos $B(x)$.

Definición 2.2. La especificación de un PTS es una tripla $\mathcal{S} = (\mathcal{S}, \mathcal{A}, \mathcal{R})$, donde:

1. $\mathcal{S}$ es un conjunto de sorts.

2. $\mathcal{A} \subseteq \mathcal{S} \times \mathcal{S}$ es un conjunto de axiomas.

3. $\mathcal{R} \subseteq \mathcal{S} \times \mathcal{S} \times \mathcal{S}$ es un conjunto de reglas.

A diferencia de los sistemas de tipos estudiados hasta ahora, existe una ligera disparidad en el contexto presente entre el lambda cálculo simplemente tipado y uno dependientemente tipado, y es que, primero, los contextos ahora pueden tener presencia de variables de tipos, de la forma $A : s$ con $s : \mathcal{S}$. Segundo, el orden en el que se insertan juicios $A : s$ en Γ importa para poder manejar de manera correcta y consistente las dependencias, por ejemplo, un juicio de la forma $A : s, \sigma : A$ es totalmente válido, pero $\sigma : A, A : s$ carece de sentido, pues no existe un juicio previo a $\sigma : A$ del que sea posible derivar que A es un tipo bien formado.

Como se ha mencionado ya con anterioridad, la extensión a la que se dedicará el estudio del presente trabajo sera λP, sin embargo, se invita al lector a revisar con más detalles el resto de teorías que, con bastante precisión Barthe and Coquand (2002) abordan sobre el Cubo Lambda de Barendregt (Barendregt, 1991).

Definición 2.3. Sea $S = \{\star, \Box\}$ de sorts y el conjunto $\mathcal{A} = \{(\star : \Box)\}$ de axiomas. Es posible obtener λP extendiendo el conjunto de reglas, de forma que $\mathcal{R} = \{(\star, \star), (\star, \Box)\}$. Permitiendo λP introducir los axiomas $\{\star, \Box\}$ es posible definir los tipos dependientes, por tanto $B : \star$ podria contener una subexpresión $M : A : \star$.

La importancia o interés de estudio de los tipos dependientes radica en cómo estos habilitan al entendimiento de la correspondencia entre las proposiciones como tipos y los programas como

pruebas a la luz de la lógica de predicados. El conjunto de reglas definidas en la definición 2.3 nos muestra que es posible formar expresiones que habitan en $\star$ y así obtener como resultado una expresión que, también, habita en $\star$. De forma intuitiva, esto corresponde a una función ordinaria del tipo $\mathbb{N} \to \mathbb{N} : \star$, donde el dominio es de sort $\star$, y el codominio de sort $\star$. Adicional a ello, el segundo miembro de la tupla, nos indica que podemos formar una expresión que habita en $\star$ y como resultado, obtenemos una que habita en $\square$. Esto último corresponde a la posibilidad de derivaciones como $\Pi x : A.B(x)$ cuando $A : \star$ y $B : \square$, siendo B un tipo indexado por A. Por tanto, la siguiente derivación es posible:

$$\frac{\Gamma \vdash A : \star \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Vec } A\ n : \star}\ \text{Vec-Form} \tag{2.17}$$

donde $Vec\, A\, n : \star$ habita en $\square$.

Dado que los tipos pudiesen contener términos, una noción de igualdad definicional es necesaria en la teoría, por ejemplo, conviene considerar los términos $0 : \mathbb{N}$ y $0 + 0 : \mathbb{N}$ como definicionalmente iguales y, por tanto, no habría distinción entre *Vec(0)* y *Vec 0 + 0*. Si *M : Vec(0)* y *N: Vec 0 + 0*, nos lleva a un juicio de la forma $M = N : \sigma$ que se lee *M y N son términos definicionalmente iguales en el tipo* σ.

Antes de detallar los constructores que conforman el lenguaje, conviene fijar el esquema con el que cada uno de ellos se presenta. Siguiendo a Hofmann (2009), un tipo queda determinado por cuatro clases de reglas.

- Reglas de formación, que especifican cómo formar el tipo.
- Reglas de introducción, que especifican cómo formar términos del tipo.
- Reglas de eliminación, que especifican cómo usar términos del tipo.
- Reglas de cómputo, que especifican cómo las reglas de eliminación actúan sobre los términos introducidos.

Este mismo esquema se conserva en la extensión con tipos cociente del capítulo 6, donde a cada regla se le hace corresponder la ecuación de Maude que la realiza.

2.3.1. Tipo Π

Con la intuición anterior, se presentan las reglas del tipo Π. La regla de formación recoge que $\Pi x : A.\, B$ es un tipo cuando A lo es y B lo es en el contexto extendido con $x : A$, de modo que B puede depender de x:

$$\frac{\Gamma \vdash A : \star \quad \Gamma, x : A \vdash B : \star}{\Gamma \vdash \Pi x : A.\, B : \star}\ \Pi\text{-Form} \tag{2.18}$$

La introducción es la abstracción lambda, que habita el tipo Π construyendo un término b de tipo B bajo la hipótesis $x : A$:

$$\frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x : A.\, b : \Pi x : A.\, B}\ \Pi\text{-Intro} \tag{2.19}$$

La eliminación es la aplicación. Al aplicar un término de tipo $\Pi x : A.\, B$ a un argumento $a : A$, el tipo del resultado es B con a sustituido por x, lo que refleja aquí la dependencia del codominio respecto al dominio:

$$\frac{\Gamma \vdash f : \Pi x : A.\, B \quad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[x := a]}\ \Pi\text{-Elim} \tag{2.20}$$

Finalmente, la regla de cómputo es la β-reducción, que describe cómo interactúan introducción y eliminación, donde aplicar una abstracción a un argumento reduce a la sustitución de dicho argumento en el cuerpo:

$$\frac{\Gamma, x : A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda x : A.\, b)\ a \equiv b[x := a] : B[x := a]}\ \Pi\text{-Comp} \tag{2.21}$$

Definición 2.4 (Igualdad definicional)**.** Decimos que dos términos son definicionalmente iguales cuando reducen, luego de cero o más pasos de reducción, a un término en común (Mimram, 2020). La igualdad a la que hacemos referencia en este caso, no es igualdad proposicional, denotada con $=$, sino, a la igualdad que es inherente del lenguaje base $\equiv$ que corresponde a la equivalencia de términos bajo β-reducción.

```
1  op _+_ : Nat Nat -> Nat .
2  eq 0 + N = N .
3  eq s(N) + M = s(N + M) .
```

En este ejemplo definimos la operación de la suma, donde el operador + recibe dos argumentos de sort Nat y retorna un elemento del mismo sort. Las ecuaciones siguientes definen su comportamiento de forma recursiva sobre el primer elemento. La primera ecuación cubre el caso base, donde el primer argumento es 0 y, por tanto, se reduce a N; la segunda tiene la forma $s(N) + M$ evaluando de forma recursiva a $s(N + M)$ En este tipo de igualdad el lenguaje no puede discernir de dos términos, pues son exactamente iguales por definición. Por ejemplo, es posible afirmar que $0 + N$ y N son definicionalmente iguales dada la definición inductiva de la suma que en Maude, podemos proponer que $0 + N$ reduce a N en un solo paso, por tanto, es posible usar $0 + N$ y N de forma intercambiable a lo largo del programa.

Sin embargo, esto no siempre es el caso: en contraste $N + 0$, que no es definicionalmente igual a N porque *no existe* una ecuación en nuestra definición de la suma que inmediatamente pueda verificar que, en este caso, $0 + N$ sí reduce a N. Pero sí que podemos demostrar proposicionalmente, mediante una prueba más elaborada por inducción en N, de que esta igualdad sí se cumple y, finalmente, concluir que $N + 0 \equiv N$.

Definición 2.5 (Igualdad proposicional)**.** Decimos que dos términos son proposicionalmente iguales cuando existe un término p que habita $\mathrm{Id}_A(a, b)$, es decir, existe una prueba $p : Id(a, b)$ como testigo de esta igualdad en A. A diferencia de la igualdad definicional, que se obtiene en términos de β-reducción y es una igualdad inherente del lenguaje, la igualdad proposicional debe ser demostrada explícitamente mediante la construcción de un testigo, esto es, un término p que es prueba de dicha igualdad. El constructor canónico del tipo identidad es $\mathrm{refl}_a : \mathrm{Id}_A(a, a)$, que atestigua que todo término es trivialmente igual a sí mismo. Para demostrar igualdades no triviales, se requiere construir un término del tipo correspondiente, tipicamente, por inducción.

Ejemplo 2.12. Considerando el mismo ejemplo de la suma sobre naturales, definida por recursion sobre el primer argumento, la proposicion $0 + n \equiv n$ es una igualdad definicional, pues el

término $0 + n$ reduce directamente a n por la primera clausula, de modo que refl_n basta como testigo. En cambio, la proposicion $n + 0 \equiv n$ no es una igualdad definicional, ya que cuando n es una variable, la expresion $n + 0$ no puede reducirse porque Maude no encuentra una ecuacion que inmediatamente pueda cubrir este caso. Es necesario construir el testigo por inducción sobre n:

- El caso base, donde si $n = 0$, entonces $0 + 0$ reduce a 0, y refl_0 es el testigo.
- El caso inductivo, donde si $n \coloneqq \text{suc}(k)$, entonces $\text{suc}(k) + 0$ reduce a $\text{suc}(k + 0)$ por la segunda clausula. Aplicando cong_{suc} a la hipótesis inductiva $p_k : k + 0 \equiv k$, obtenemos el testigo $\text{cong}_{\text{suc}}(p_k) : \text{suc}(k + 0) \equiv \text{suc}(k)$.

En Maude, podemos simular este comportameinto de la siguiente manera, asumiendo que el tipo de proposición es un sort cualquiera.

```
1  sort A B .
2  subsort A B < Nat .
3  sort Proof .
4  op cong : (A -> B) Proof -> Proof .
5  op refl : -> Proof.
6
7  op zero' : Nat -> Proof .
8  eq zero'(0) = refl .
9  eq zero'(s(N)) = cong(zero'(N))
```

La existencia de esta prueba es lo que nos permite concluir que $n+0 \equiv n$ proposicionalmente.

Corolario 2.4. *Dos términos que son definicionalmente iguales, también lo son proposicionalmente. Lo contrario no tiene por qué cumplirse.*

2.3.2. Tipo Σ

Los sistemas de tipos puros, en principio, no proveen constructores más allá del tipo Π, por tanto, el próximo constructor o tipo al que dedicarémos un breve acercamiento, que también formará parte de la teoria y lenguaje que se formalizará en un capitulo posterior, es el tipo sigma, denotado con Σ. Representado en el lambda cálculo simplemente tipado como $A \times B$, es el tipo de una tupla en el que el primer y segundo componente son de tipo A y B respecivamente, sin embargo, en los tipos dependientes, el segundo componente B pudiese tener una ocurrencia x de tipo A, luego, su tipo, tiene la signatura: $\Sigma x : A.B$, o bien $\langle M, N \rangle_{\Sigma x:A}$, donde M habita en A y N en $B(N)$. Para facilitar o, más bien, asegurar la decibiliad del prototipo, este tipo estará acompañado de su anotación.

El conjunto de pseudo-términos $\mathcal{T}$, se extiende, por tanto, con un nuevo miembro representando al tipo sigma:

$$\mathcal{T} = ... \mid \Pi\mathcal{V} : \mathcal{T}.\mathcal{T} \mid \langle \mathcal{T}, \mathcal{T} \rangle \mid fst\,\mathcal{T} \mid snd\,\mathcal{T}$$

Donde *fst* y *snd* son *proyecciones* que extraen el primer y segundo componente de la tupa $\langle \mathcal{T}, \mathcal{T} \rangle$. Sus reglas de formación y eliminación, plantea Barthe and Coquand (2002), se definen como sigue:

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B[x := M] \quad \Gamma \vdash \Sigma x : A.B : \star}{\Gamma \vdash \langle M, N \rangle_{\Sigma x:A.B} : \Sigma x : A.\, B}\ \Sigma\text{-form} \tag{2.22}$$

$$\frac{\Gamma \vdash M : \Sigma x : A.\, B}{\Gamma \vdash \text{fst}\ M : A}\ \Sigma\text{-fst} \tag{2.23}$$

$$\frac{\Gamma \vdash M : \Sigma x : A.\, B}{\Gamma \vdash \text{snd}\ M : B(\text{fst}\, M)}\ \Sigma\text{-snd} \tag{2.24}$$

Seguidamente se describen las reglas de cómputo, que fijan el comportamiento de las proyecciones sobre una tupla concreta. Al proyectar sobre $\langle M, N \rangle$, la primera componente reduce a M y la segunda a N:

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B[x := M]}{\Gamma \vdash \text{fst}\, \langle M, N \rangle_{\Sigma x:A.\, B} \equiv M : A}\ \Sigma\text{-comp-fst} \tag{2.25}$$

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B[x := M]}{\Gamma \vdash \text{snd}\, \langle M, N \rangle_{\Sigma x:A.\, B} \equiv N : B[x := M]}\ \Sigma\text{-comp-snd} \tag{2.26}$$

Estas dos igualdades son las que, en el prototipo, realizan las funciones auxiliares normFst y normSnd al reducir la proyección sobre un par, según se detalla en el capítulo 5.

Los tipos sigma serán utiles para probar propiedades del lenguaje en la práctica y, posteriormente, comprobar que la formalización de la teoria de tipos estudiada en este trabajo cumple con su función.

2.4. Tipos cociente

Los cociente son un término ubicuo en matemáticas, sin embargo, desde la teoría de conjuntos están presentes y se extiende dicho concepto con naturalidad a la teoría de tipos. En los conjuntos, es posible definir el cociente de un conjunto $\mathbb{S}$ en clases de equivalencia, obteniendo el cociente del conjunto, donde cada elemento que lo conforma es considerado igual, o equivalente, dada una relacion de equivalencia $\sim$. De forma análoga, es posible definir el cociente de un tipo, donde dada una relación de equivalencia, los elementos son iguales dentro del cociente pero, fuera de ellos, aún es posible distinguir uno del otro. Es posible afirmar, por ende, que los tipo cociente nos permiten redefinir la igualdad proposicional en un sistema de tipos.

Los tipos cociente son el concepto clave del presente trabajo, puesto a que son sumamente importantes en la formalización de conceptos matemáticos y el razonamiento de programas de lenguajes funcionales. Los cocientes, son un concepto de la teoría de tipos extensional. Las alternativas que la teoría intensional ofrece para simularlos se discuten en capítulo 3.

Definición 2.6. Una relación de equivalencia $\sim$ en un conjunto A es una relación binaria de signatura $R : A \to A \to \mathit{Prop}$ que cumple las siguientes propiedades:

1. Reflexividad: $\forall a \in A,\ a \sim a$.
2. Simetría: $\forall a, b \in A,\ a \sim b \to b \sim a$.
3. Transitividad: $\forall a, b, c \in A,\ a \sim b \to b \sim c \to a \sim c$.

Por tanto, la clase de equivalencia de un elemento $a \in A$ se denota $[a]$, donde $[a] = \{x \in A \mid a \sim x\}$. El conjunto de clases de equivalencia de A viene denotada como A/R, que se lee *el cociente de A módulo R* o *el cociente de A por R*. De forma intuitiva, un tipo cociente A/R es un tipo donde su igualdad ha sido redefinida por una relación de equivalencia $\sim$.

Es necesario mencionar la utilidad práctica de los cocientes y, dejar claro que no son un concepto puramente matemático y realmente pueden ser de utilidad en el desarrollo de programas y que a parte de ello permiten razonar sobre las propiedades y relaciones de los términos en el mismo. Tómese como ejemplo el caso de los naturales representados como un par (a, b), donde con su diferencia $a - b$ permite identificar que muchos de ellos pueden representarse como un mismo entero, donde, (3, 1), (4, 2) y (5, 3) representaría el numero 2 y ser, de alguna forma, indistiguibles para el programa. La condición que los agrupa, $a + b = b + c$ es en este caso la relación de equivalencia, y formar el cociente dada esta relación es lo que nos permite que colapsen estos elementos en un solo valor, $[t]$, conocido como *representante*, y como consecuencia los elementos que agrupa resultan indiscernibles para el sistema de tipos.

Los multiconjuntos o *Set* siguen el mismo patrón. Recuérdese estos como listas en las que el orden y la repetición de los elementos que la conforman carecen de importancia alguna. Partimos, pues, del tipo de las listas y cocientamos por la relación que identifica dos de ellas cuando una es permutación de la otra. Por ejemplo, dadas dos listas l_1 y l_2, donde una es permutación de la otra y ambos términos son completamente distintos:

$$\begin{aligned} l1 &= cons(1, cons(2, cons(1, nil))) \\ l2 &= cons(2, cons(1, const(1, nil))) \end{aligned} \tag{2.27}$$

La evaluación *TyEq(t1, t2) = false*, correctamente, dado que ambas formas normales no coinciden. Sin embargo, de cocientar sobre el tipo lista de naturales *List Nat / R* conseguiríamos relacionar ambas cuando una es permutación de la otra, como es el caso. Por tanto, la relación solo es cierta de cumplirse:

$$l1 \sim l2 = \forall x.(x, count(x, l1)) = (x, count(x, l2)) \tag{2.28}$$

Una vez definido el cociente, obtendríamos $[l1]$ y $[l2]$ que dentro de *List Nat / R* designan al mismo elemento $const(1, cons(1, cons(2, nil)))$:

$$qax(l1, l2, [ListNat/R], p) : Id(ListNat/R, [l1], [l2]). \tag{2.29}$$

qax denota que pese a la diferencia sintáctica de ambos términos, hay una igualdad proposicional entre ellos y existe una prueba p de ello, esto es, el tipo Id de igualdad entre $[l1]$ y $[l2]$ dentro del cociente.

En definitiva, desde el lado de la programación, disponer de cocientes amplía el repertorio de tipos de datos algebraicos y facilita razonar sobre tipos dependientes, nociones de igualdad y sobre la semántica de programas concurrentes, como sugieren Li (2014) y Hofmann (1995).

Según lo estudiado por Li (2014), las reglas de formación de los tipos cociente se definen como sigue:

$$\frac{\Gamma \vdash A : \star, \quad \Gamma x, y : A \vdash x \sim y : Prop}{\Gamma \vdash A/\sim} \text{ Q-Form} \tag{2.30}$$

Donde $A/\sim$ es el tipo cociente, dado que A es un tipo bien formado en Γ, x y y habitan en A, y existe una prueba, de tipo $Prop$ que atestigua que $x \sim y$.

Adicionalmente, la regla $Q - intro$ permite inferir que $[a]$ es una clase de equivalencia de tipo $A/\sim$, dado que $a : A$ en Γ

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash [a] : A/\sim} \text{ Q-Intro} \tag{2.31}$$

La regla $Q - Ax$, indica que dada una prueba p de equivalencia de $a \sim b$, y dos elementos a y b de tipo A, nos es posible definir que $[a]$ y $[b]$ son iguales definicionalmente dentro del cociente $A/\sim$, pero, fuera de el, son elementos totalmente distinguibles por el sistema de tipos.

$$\frac{\Gamma \vdash a, b : A \; \Gamma \vdash p : a \sim b}{\Gamma \vdash Qax(p) : [a] =_{A/\sim} [b]} \text{ Q-Ax} \tag{2.32}$$

Además de ello, se propone una regla de eliminación que permite aplicar una función f en $A/\sim$, a la que nos referirémos como *lifting*, si y solo si, la misma respeta la relación de equivalencia (Hofmann, 1995).

$$\frac{\begin{array}{c}\Gamma \vdash B \qquad \Gamma \vdash f : A \to B \\ \Gamma, a, b : A, p : a \sim b \vdash f^{\sim}(a, b, p) : f(a) =_B f(b) \qquad \Gamma \vdash q : A/\sim\end{array}}{\Gamma \vdash \hat{f}(q) : B} \text{ (Q-elim)} \tag{2.33}$$

La regla de cómputo $Q - comp$ indica que al aplicar f (asumiendo que f es una función que respeta la equivalencia) a un representante $[a]$ del cociente, obtenemos $f(a)$.

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{Qcomp}(a) : \hat{f}([a]) = f(a)} \text{ Q-comp} \tag{2.34}$$

El resto de reglas estudiadas por Li (2014) se omitirán, pues se consideran fuera del alcance del prototipo.

Capítulo 3

Estado del arte

3.1. Cocientes en teoría de tipos intensional

En un principio, Martin Löf propuso dos variantes de la teoría de tipos que se diferencian por el tratamiento que dan a la igualdad: la teoria de tipos intencional, y la teoria de tipos extensional. La mayoria de lenguajes de programación de hoy en día, parten de una teoría de tipos intencional porque sus propiedades son decidibles, donde en la teoría de tipos extensional no se garantiza del todo. La razón de esto, es que en la teoría de tipos intensional, la igualdad proposicional definida por tipos de identidad no implica igualdad definicional, y la verificación de tipos es decidible. En la teoría de tipos extensional, por otro lado, la igualdad proposicional se identifica con la igualdad definicional, lo que hace que la verificación de tipos sea indecidible (Li, 2014).

Debido a sus buenas propiedades computacionales, la teoría de tipos intensional es más popular, sin embargo, carece de algunos conceptos extensionales importantes, como la extensionalidad funcional y los tipos cociente, que estaremos abordando.

Muchos asistentes de pruebas y lenguajes con tipos dependientes como Coq, Agda o Lean se apoyan en la teoría intensional. Al ser decidible su igualdad definicional, la verificación de tipos se puede automatizar y todo programa bien tipado es terminante, que son justo las garantías que se espera de un lenguaje con el que se pretende razonar de forma fiable o, mejor dicho, sensata. Lo que se paga a cambio es la ausencia de los conceptos extensionales que venimos comentando: la extensionalidad funcional, la identificación de pruebas distintas de una misma proposición y los tipos cociente (Li, 2014).

Podría pensarse que basta con postular estos conceptos como axiomas, pero esa vía tiene un coste, y es que un axioma se limita a afirmar que cierto término existe, sin decir cómo se calcula, y esa falta de garantía es lo que imposibilita al sistema de tipos determinar si una determinada prueba de igualdad existe o no. Como consecuencia de esto, pues, obtenemos términos cerrados que ya no reducen a ningún valor concreto, con lo que perdemos la canonicidad y, con ella, la razón de haber elegido la teoría intensional desde el principio.

En la teoria de tipos intensional, como se mencionó, no existe la noción de tipos cociente dada su naturaleza de redefinir la igualdad definicional, sin embargo, los setoides se han implementado para capturar el mismo comportamiento.

Los setoides, sugieren Li (2014) y Hofmann (1995) definidos como $(A, \sim)$, están compuestos por:

1. Un tipo $A : Type$.
2. Una relación binaria $\sim$: $A \to A \to Prop$.
3. Una prueba p de que una función f respeta la equivalencia (es decir, que es reflexiva, transitiva y simetrica).

Una función $f : A \to B$ respeta la equivalencia $\sim$ si $\forall(x, y : A) \to x \sim y \to f(x) =_B f(y)$.

Sin embargo, en la práctica el modelo de los setoides no resulta del todo viable, porque cada función debe ir acompañada de una prueba de que respeta la equivalencia $\sim$ correspondiente, y estas pruebas de coherencia han de construirse y transportarse manualmente en cada composición, lo que vuelve el desarrollo inmanejable y tedioso.

La alternativa que ha resultado viable, consiste en dejar intacta la sintaxis y buscar interpretaciones donde los tipos sean algo más que conjuntos. En esa línea aparecieron durante los años noventa varios modelos, entre ellos los modelos de setoides de Hofmann y de Altenkirch, y el modelo de grupoides de Hofmann y Streicher (Hofmann, 1995). Leer los tipos como grupoides abrió una puerta que otros siguieron hacia estructuras de dimensión cada vez mayor, y en esa misma dirección se sitúa la teoría homotópica de tipos propuesta por Voevodsky, donde los tipos se piensan como espacios, los términos como puntos en dichos espacios y los tipos de identidad como caminos entre ellos. Bajo esa lectura la igualdad pasa a ser un tipo con estructura propia en cada nivel.

3.2. NuPRL y la teoría de tipos extensional

La noción de tipos cociente, tiene sus origenes en NuPRL, el cual es un sistema de verificación, desarollo de pruebas y formalización de matemáticas que extiende la teoría de tipos de Martin-Lof. Su sistema de tipos y como es tratada la igualdad difiere en gran parte de la mayoría de otras teorías empleadas por herramientas similares. En Coq, que está basado originalmente en el Cálculo de Construcciones, existe una sola relación de igualdad para todos los tipos, y no hay una noción de igualdad específica para un tipo en concreto (Nogin, 2002).

Por otro lado, en NuPRL cada tipo integra su propia relación de equivalencia, y su sistema de tipos garantiza que cualquier término bien tipado respete esa equivalencia. Asimismo, la formalización de tipos cociente en NuPRL se ve limitada porque no permite tomar plena ventaja de la extensionalidad, dado que los cocientes en su sistema asumen que el predicado de igualdad es, de hecho, intensional (Constable et al., 1986).

3.3. Tipos cociente en Lean

Lean es un lenguaje dependientemente tipado y probador de teoremas relativamente nuevo que posee un sistema de tipos bastante robusto, permitiendo formalizar conceptos matemáticos, probar fórmulas y propiedades de los lenguajes. En los tipos cociente en Lean, todo elemento relacionado bajo $\sim$ es considerado igual, esta relación es dada por una instancia de *Setoid* que el lenguaje expone como un tipo concreto de forma interna, luego la igualdad es respetada de

forma universal por el sistema de tipos. Por tanto, Lean no puede discernir entre dos elementos que son considerados iguales bajo esta relación. Su tipo viene dada por la siguiente signatura en Lean:

```
Quotient.{u} : { : Sort u}    Setoid       Sort u
```

Donde *Quotient* construye el cociente dado un *Setoid*. De manera interna, *Setoid* está conformado por un tipo α, una relación $r : \alpha \to \alpha \to Prop$ y una prueba de que r es una relación de equivalencia. Intuitivamente, podría decirse que *Setoid* y *Quotient* son fundamentalmente lo mismo, y en parte lo son, pero *Setoid* no provee las mismas garantías y coherencia que *Quotient*, por lo que sería posible aplicar una función f que no respetase $\sim$ sobre algún representante. Además de esto, en los setoides dada la relación $a \sim b$ en un tipo A, no implica igualdad proposicional, a diferencia de los cocientes, donde la igualdad proposicional viene dada por el axioma:

```
Quotient.sound.{u} { : Sort u} {s : Setoid } {a b : } :
  a     b     Quotient.mk s a = Quotient.mk s b

Quotient.ind.{u} { : Sort u} {s : Setoid }
  {motive : Quotient s   Prop} :
  (    (a :  ), motive (Quotient.mk s a))      (q : Quotient s), motive q
```

Sound sostiene que si dos elementos son equivalentes dada una relación de equivalencia, entonces son iguales dentro del cociente. Además, *ind*, indica que para probar que una proposición es cierta para todo elemento del cociente, basta solo con probar que es cierta para todos sus representantes. Este último es el principio de inducción para el tipo de identidad.

En el sistema de tipos de Lean, es suficiente con demostrar que dos elementos del tipo subyacente están relacionados por la relación de equivalencia para demostrar que son iguales en el cociente. Sin embargo, esta es igualdad proposicional y no definicional, los representantes se vuelven equivalentes con el axioma *Quotient.sound*, pero siguen siendo diferentes por definición. (The Lean Language Reference Contributors, 2025).

En definitiva, Lean provee un sistema de tipos sumamente sofisticado y robusto para formalizar y permitir el razonar sobre programas. El tipo *Quotient* que implementa de forma interna, constituido en principio por Setoids, provee una referencia de suma importancia para entender los fundamentos y antecedentes de este tipo de abstracción en lenguajes actuales.

3.4. Formalización de sistemas de tipos en lógica de reescritura

La capacidad de Maude para definir sintaxis con notación definible por el usuario, junto con su soporte para reescritura y para reescritura no determinista, lo convierten en una plataforma especialmente apropiada para formalizar lenguajes de programación y sistemas de tipos. La idea general es trasladar la sintaxis abstracta del lenguaje a una signatura Σ, las reglas de tipado o de evaluación de la semántica operacional a reglas de reescritura R, y las identidades estructurales sobre los términos, como la α-equivalencia o la igualdad definicional, a ecuaciones E. Esta metodología ha sido aplicada con éxito a lenguajes reales, desde fragmentos de C hasta Java o JavaScript, dando lugar a prototipos no solo ejecutables, sino también analizables con las herramientas del entorno formal de Maude (Durán et al., 2020).

La lógica de reescritura provee la semántica para formalmente especificar lenguajes de programación como teorías de reescritura, y los precedentes de esto, sugieren que en la práctica

ha sido estudiada ampliamente y se ha puesto en práctica durante mucho tiempo. Las mismas describen las ventajas de las especificaciones como ecuaciones y la definición de la semántica de prototipos de lenguaje, dado que estas especificaciones son ejecutadas en Maude, se convierten en interpretes nativos del lenguaje que se intenta formalizar. Una especificación de Maude para la semántica de Scheme fue empleado por la Universidad de Illinois que, como resultado, fue un 75 por ciento más rapido que el interprete nativo del lenguaje. Adicional a ello, especificaciones para subconjuntos de un lenguaje tipo C y ML también han sido discutidos en detalle.

La razón por la que la lógica de reescritura provee un excelente marco de trabajo para el propósito de describir e implementar reglas de inferencia y tipado, es que estas reglas pueden ser axiomatizadas y reinterpretadas por una teoría ecuacional y, posteriormente, entendidas como reglas de reescritura (Clavel et al., 2007).

Una contribución de suma importancia en la representación de sistemas de tipos con Maude viene dada por Stehr and Meseguer (2004), donde se muestra la representación de un sistema de tipos puro a la lógica de reescritura, concretamente, el *Open Calculus of Constructions*, y que además, argumentan, Maude es un lenguaje sólido para la formalización de una familia más extensa de sistemas de lógica de orden superior, y cómo es posible razonar sobre las reglas de inferencia en términos de la lógica de reescritura.

Open Calculus of Constructions es una extensión del Cálculo de Construcciones con tipos dependientes y universos, que incorpora lógica de reescritura y lógica ecuacional de pertenencia. Parte del enfoque de Stehr and Meseguer (2004) radica en implementar los juicios de tipado como operadores mixfix y determinar si los mismos son derivables, de la forma:

```
1 op _|-_:_ : Context Trm Trm -> Judgement .
2 sort Derivable .
3 subsort Derivable < Judgement .
4 cmb (emptyContext |- s1 : s2) : Derivable if (s1,s2) : Axioms .
5 cmb (G,(X : A) |- X : A) : Derivable if
6   (G |- A : s) : Derivable /\ not(X in G) .
```

La correspondencia entre las reglas de inferencia y la lógica de reescritura es mucho más clara en esta aproximación. La pertenencia es definida mediante *cmb* que condiciona si un término pertenece, o no, al sort *Derivable* y a partir de allí formar un juicio. Además, las sustituciones en el cálculo mencionado evitan la captura de variables implementando las operaciones *shift* y *lift* acorde a la representación de De Brujin, que puede ser consultada con más detalle en Pierce (2002). Argumentan Stehr and Meseguer (2004), también, las razones por las cuales esta especificación es mucho más natural que la de lenguajes de programación funcionales, y es que la semántica operacional de las ecuaciones del cálculo de sustituciones tratado en su trabajo puede visualizarse fácilmente como reglas de reescritura. En definitiva, el prototipo presentado por Stehr and Meseguer (2004) puede ser usado como lenguaje de programación y como asistente de pruebas, demostrando la eficacia de Maude para explorar familias de teorías de tipos avanzadas como lo son las variantes del cálculo de construcciones y razonar sobre los tipos dependientes.

Junto con lo anterior existe un precedente que relaciona la teoría de tipos con la reescritura directamente. Esta conexión radica en que Blanqui (2001) estudia el sistema formal *Type System Modulo*, donde la relación de conversión es generada particularmente por una relación $\rightarrow_{\mathcal{R}}$. A partir de este marco, define los *Reduction Type System*, en los que la conversión es inducida por una relación de reducción. Posteriormente en su investigación, examina el caso de los *Algebraic Type Systems*, un clase particular de *RTS* cuya relación de reducción está definida en términos de *β-reducción* y reglas de reescritura. Sugiere Blanqui que la introducción de la reescritura adopta

relevancia en un sistema de tipos dependiente, puesto que permite tipar términos que no pueden ser tratados únicamente mediante la convertibilidad propia del cálculo y, en consecuencia, amplía las proposiciones que pueden formalizarse. Formalmente, esta limitación se refleja en la regla de conversión:

$$\frac{\Gamma \vdash t : T, \quad T \equiv_\beta T'}{\Gamma \vdash t : T'} \text{ Conv} \tag{3.1}$$

Donde se establece que, si dado un término $t : T$, donde T es *β-convertible* a T', entonces es posible afirmar que $t : T'$. Nótese que la regla de conversión es aplicada en términos de igualdad definicional. Blanqui propone el ejemplo de dos listas indexadas por su longitud $list((n+n')+n'')$ y $list(n' + (n' + n''))$, donde se puede comprobar que ambas son definicionalmente iguales por asociatividad de la suma, aunque ambos términos no son *beta-convertibles*, por tanto no es posible concluir que $list((n + n') + n'')$ es inmediatamente reemplazable por $list(n + (n' + n''))$. Como solución a esta limitante, Blanqui incorpora una relación de reescritura $\to_\mathcal{R}$, generada por un conjunto de reglas arbitrarias $\mathcal{R}$ definidas por el usuario:

$$\frac{\Gamma \vdash t : T \quad T \leftrightarrow^*_{\beta\mathcal{R}} T' \quad \Gamma \vdash T' : \star}{\Gamma \vdash t : T'} \text{ Conv} \tag{3.2}$$

De esta forma, igualdades que anteriormente debían tratarse como pruebas explícitas pueden incorporarse al mecanismo de conversión del sistema. Sin embargo, permitir reglas de reescritura arbitrarias dentro de la conversión introduce un inconveniente, y es que es necesario garantizar que la reducción no destruye las propiedades metateóricas necesarias para que el sistema de tipos sea utilizable, entre ellas, la consistencia y decibilidad de la verificación de tipos.

Blanqui además señala que la reescritura permite modelar tipos cociente orientando las ecuaciones que caracterizan la relación de equivalencia como reglas de reescritura. De manera que $p(s(x)) = x$ y $s(p(x)) = x$ pueden replantearse como $p(s(x)) \to x$ y $s(p(x)) \to x$ respectivamente. Cuando el sistema resultante es confluente y fuertemente normalizante, cada clase de equivalencia admite una forma normal única, en este caso x, que actúa como representante canónico.

En este marco, la principal contribución de Blanqui consiste en establecer condiciones generales que garantizan la normalización fuerte del *Calculus of Constructions* extendido con reescritura a nivel de tipos. En particular, para el *Calculus of Algebraic Constructions*, dichas condiciones aseguran que la reducción combinada $\to_\beta \cup \to_\mathcal{R}$ preserva el tipado y es fuertemente normalizante. Asimismo, Blanqui demuestra que este marco comprende un subsistema significativo del *Calculus of Inductive Constructions* y a *Natural Deduction Modulo*, incorporando reescritura de primer y orden superior dentro de la relación de conversión

La conexión entre este marco y el presente trabajo radica en que el prototipo desarrollado constituye una implementación práctica del $\lambda\Pi$-cálculo módulo reescritura (Blanqui, 2001). Específicamente, la regla de computación del cociente se modela orientando la reducción del eliminador hacia el representante como una regla de reescritura de la forma $(f, A, [t]) \to_\mathcal{R} f(t)$. Al implementar esta reducción en el normalizador, la conversión de nuestro sistema se determina incorporando esta regla ecuacional durante la verificación bidireccional de tipos, garantizando de forma integrada la decidibilidad del chequeo de tipos y la preservación de los mismos, de acuerdo a los criterios sugeridos por Blanqui (2020).

Capítulo 4

Maude y la lógica de reescritura

La reescritura de términos constituye, a la luz de los capítulos anteriores, una generalización natural de la β-reducción del λ-cálculo. Donde antes orientábamos un redex de la forma $(\lambda x.\, M)\, N$ hacia $M[x := N]$, ahora disponemos de un conjunto arbitrario de ecuaciones $l = r$, leídas como reglas $l \to r$, y las aplicamos a un término siempre que en él aparezca una instancia del lado izquierdo. El proceso de cómputo se reduce, entonces, a buscar redexes y a contraerlos, exactamente igual que antes, pero en un marco mucho más flexible que permite especificar tipos de datos, funciones y, como veremos, sistemas concurrentes enteros.

Maude es un lenguaje declarativo de alto rendimiento, diseñado y mantenido a lo largo de más de dos décadas por un consorcio en el que participan SRI International, la Universitat Politécnica de Valéncia, la Universidad Complutense de Madrid, la Universidad de Málaga, la Universidad de Illinois en Urbana-Champaign y la Universidad de Stanford (Durán et al., 2020; Clavel et al., 2007). Su fundamento teórico es la lógica de reescritura, que admite simultáneamente dos lecturas. Por un lado, es un marco semántico en el que pueden especificarse de modo natural una amplia variedad de sistemas concurrentes, desde redes de Petri hasta cálculos de procesos. Por otro, es un marco lógico en el que se pueden codificar otras lógicas tomando sus reglas de inferencia como reglas de reescritura. La máxima que sintetiza esta dualidad, y que articula buena parte del diseño del lenguaje, es la equivalencia

$$\text{Cómputo} \;=\; \text{Deducción}$$

de modo que un programa Maude no es otra cosa que una teoría de reescritura, y la ejecución del programa coincide con la búsqueda de derivaciones en dicha teoría (Clavel et al., 2007). Esta correspondencia es la que hace de Maude un candidato especialmente adecuado para implementar prototipos ejecutables de sistemas de tipos, pues la sintaxis abstracta del lenguaje que se quiera modelar y sus reglas de tipado pueden trasladarse de manera casi directa a Maude.

4.1. Sorts y lógica ecuacional ordenada

La primera diferencia notable respecto del λ-cálculo es que Maude obliga a tipar los términos desde la propia signatura. Cada constante y cada operación declara explícitamente los sorts de sus argumentos y de su resultado, donde por *sort* entendemos lo que en la teoría de tipos llamaríamos un tipo base. La idea de organizar el universo de discurso en sorts proviene de la lógica many-sorted, una variante de la lógica de primer orden en la que el dominio se descompone en una colección indexada de subdominios y cada símbolo de función o predicado especifica el

sort de cada uno de sus argumentos (Manzano and Aranda, 2022). Esta restricción sintáctica es muy poco distinguible, pero realmente nos permite determinar, por ejemplo, los términos aritméticos de los términos booleanos sin necesidad de codificarlos en un dominio común, y descarta en principio expresiones sin sentido como sumar un valor de verdad a un número natural.

Maude va, sin embargo, un paso más allá de la lógica many-sorted habitual y adopta la lógica ecuacional *order-sorted*. En esta variante, los sorts forman un conjunto parcialmente ordenado $(S, \leq)$ en el que la relación $s \leq s'$ se interpreta como inclusión de subsort. Por ejemplo, si declarásemos que $\mathsf{NzNat} \leq \mathsf{Nat}$, todo número natural no nulo es también, automáticamente, un número natural, sin necesidad de coerciones explícitas. Las operaciones pueden también estar sobrecargadas a lo largo de la jerarquía de subsorts, de manera que una misma suma puede declararse a su vez sobre Nat y sobre NzNat, y el sistema seleccionaría en cada caso el sort más preciso para el resultado.

4.2. Módulos funcionales

La unidad básica de especificación en Maude es el módulo. Un módulo funcional, declarado entre las palabras clave `fmod` y `endfm`, agrupa una signatura de sorts y operaciones junto con un conjunto de ecuaciones que se interpretan como reglas de reescritura de izquierda a derecha. Es la unidad análoga, en este lenguaje, a lo que en programación funcional sería un módulo con sus tipos algebraicos y sus funciones puras. Considerese el siguiente ejemplo, en el que especificamos los números naturales en notación de Peano:

```
fmod NAT-PEANO is
  sorts NzNat Nat .
  subsorts NzNat < Nat .
  op 0 : -> Nat [ctor] .
  op s : Nat -> NzNat [ctor] .
  op _+_ : Nat Nat -> Nat .
  vars N M : Nat .
  eq 0 + N = N .
  eq s(N) + M = s(N + M) .
endfm
```

El módulo declara dos sorts, NzNat y Nat, con la inclusión $\mathsf{NzNat} \leq \mathsf{Nat}$. La constante 0 se introduce con el atributo `ctor`, que la marca como constructor del tipo, y el sucesor s se declara con el mismo atributo. La suma se especifica con notación infija mediante guiones bajos, y sus dos ecuaciones recogen, casi literalmente, la definición estándar por inducción sobre el primer argumento. Una vez introducido el módulo en el intérprete, podemos solicitar la evaluación de una expresión mediante el comando `reduce`:

```
Maude> reduce s(s(0)) + s(s(s(0))) .
result NzNat: s(s(s(s(s(0)))))
```

El intérprete aplica las ecuaciones como reglas de reescritura hasta alcanzar la forma normal e infiere el menor sort posible para el resultado, en este caso NzNat.

Para que un módulo funcional tenga buenas propiedades operacionales, las ecuaciones deben ser confluentes y operacionalmente terminantes, las mismas dos nociones que estudiábamos en el capítulo del λ-cálculo, ahora reformuladas en el contexto de un sistema de reescritura arbitrario. La terminación operacional garantiza que toda secuencia de reescritura concluye en un número

finito de pasos, y la confluencia, que el resultado de la evaluación no depende del orden en que se contraen los redexes. Ambas propiedades, indecidibles en general, pueden chequearse automáticamente con las herramientas del *Maude Formal Environment* (Durán et al., 2020), entre las que destacan `MTT` para terminación y `ChC` para confluencia y suficiente completitud. Bajo estas condiciones, todo término se reduce a una forma normal única, lo que dota al lenguaje de una semántica determinista para sus módulos funcionales.

4.3. Módulos de sistema y reescritura no determinista

Junto a los módulos funcionales, Maude provee módulos de sistema, declarados entre `mod` y `endm`. Un módulo de sistema extiende a uno funcional con un conjunto adicional de reglas de reescritura $l \to r$ que modelan transiciones de un sistema, semánticamente distintas de las ecuaciones (Clavel et al., 2007). Las ecuaciones describen *cómo se calcula el estado de un término*, esto es, cómo se simplifica a su forma normal. Las reglas, en cambio, describen *cómo evoluciona el sistema*, o bien, qué transiciones puede dar de un estado a otro. La diferencia es que las reglas no se asumen confluentes ni terminantes, sino que codifican el comportamiento intrínsecamente no determinista del sistema. Un mismo término puede reescribirse de maneras distintas y conducir a estados muy diferentes, y eso es justamente lo que se quiere capturar al describir, por ejemplo, las transiciones de un protocolo de comunicación, las jugadas de un juego, o los pasos de una máquina abstracta de evaluación para un lenguaje de programación.

La semántica operacional resultante combina ambos niveles. Ante un término, Maude normaliza primero con las ecuaciones y aplica después una regla al resultado. Para explorar el espacio de estados alcanzables desde un término inicial, Maude provee los comandos `rewrite`, que sigue una traza de reescrituras, y `search`, que explora todos los estados alcanzables en anchura buscando uno que satisfaga un patrón dado (Durán et al., 2020). Sin embargo, dado que el lenguaje estará implementado dentro de un modulo funcional, `reduce` será de especial utilidad porque nos permitirá reducir las ecuaciones a su forma normal y, de este modo, cada expresión se evalúa en un único resultado final, que en Maude, se le denimonan como formas canónicas. Por definición, los resultados de las operaciones en esta álgebra son exactamente los que da el intérprete de Maude; y este es, en definitiva, el modelo computacional más preciso que se puede obtener (Clavel et al., 2007).

4.4. Formalización de lenguajes y sistemas de tipos

Para hacer visible este paradigma en el caso más elemental, considerémos un fragmento del lambda cálculo sin tipos. Especificamos los términos con tres constructores que reflejan directamente la definición inductiva propuesta en capítulos anteriores:

```
fmod LAMBDA-SYNTAX is
  protected STRING .
  sort Var .
  sort Term .
  subsort Var < Term .

  op var : String -> Var [ctor] .
  op λ_:_._ : Var Term Term -> Term [ctor prec 40] .
  op _·_ : Term Term -> Term [ctor prec 15 gather (E e)] .
endfm
```

Aquí Var es subsort de Term, lo que captura sintácticamente el hecho de que toda variable es un término. Tanto la aplicación como la lambda abstracción, son constructores que, proporcionando los argumentos que exige su signatura será posible producir un sort. Nótese los argumentos que acompañan a *ctor* en los casos de lambda abstracción y aplicación; *gather (E e)*, nos permite definir qué tipo de precedencia admitir en los constructores que preceden a la signatura del sort. Concretamente, el argumento izquierdo admite precedencia igual a la del propio operador E, y el derecho exige que sea estrictamente menor, e, en este caso, es lo que permite que $f \cdot g \cdot a$ parsee de manera correcta $f \cdot g(a)$, y luego $f(g(a))$. El símbolo de guión bajo es un espacio reservado para los argumentos de la expresión. Esto último, son operadores mixfix que nos permitirán posteriormente definir la sintaxis y gramática operacional que el núcleo del lenguaje manejará. Luego, la β-reducción puede expresarse después como una regla de reescritura en un módulo de sistema que extienda al anterior. Para evitar la captura de variables se necesitaría una operación auxiliar de sustitución, definida por ecuaciones, que renombre las variables ligadas con variables frescas cuando sea necesario. Sin embargo, omitirémos esta funcionalidad por propósitos prácticos a lo largo de esta tesis para implementar el lenguaje dependientemente tipado y, posteriormente, su extensión con tipos cociente.

Los términos de un sistema de tipos *à la Church* incluyen, junto a las variables y aplicaciones, anotaciones de tipo en cada uno de sus binders, y las reglas de tipado se expresan en forma de juicios: $\Gamma \vdash M : A$. En Maude, basta con introducir un sort para los contextos, otro para los juicios de tipado, y reglas de reescritura que codifiquen cada regla de inferencia.

Tomando como ejemplo el contexto, el mismo puede definirse como un mapeo de variables a tipos, y tipos a tipos de variable $\mathsf{x} \to \tau$, $\mathsf{Nat} \to \mathcal{U}$. El contexto por tanto, almacena información sobre los tipos bien formados en nuestro programa. De no existir un tipo o término para un tipo en Γ, decimos que es un término o tipo malformado.

$$\Gamma := \quad \emptyset \tag{4.1}$$

$$\quad | \quad (x, \tau) :: \Gamma \tag{4.2}$$

La noción de contexto puede traducirse en Maude, como hemos mencionado, creando un sort para este y las ecuaciones que le corresponden. Guiándonos, pues, de la gramática, el contexto puede bien estar vacío, o extendido. Estas operaciones, pueden ser perfectamente codificadas como muestra la siguiente algebra inicial:

```
fmod LAMBDA-SYNTAX is
  sort Ctx .
  op mt : -> Ctx [ctor] .
  op _;_:_ : Ctx Var Term -> Ctx [ctor] .
  var G : Ctx .
endfm
```

Donde la notación *_;_:_*, es un constructor para un sort de tipo *mt; var x : τ*, que representa el contexto extendido con $x : \tau$.

Tomando como ejemplo la regla de formación del tipo sigma

$$\frac{\Gamma \vdash A : \mathsf{Type} \quad \Gamma, x : A \vdash B : \mathsf{Type}}{\Gamma \vdash \Sigma\, x : A.\, B : \mathsf{Type}}\ \Sigma\text{-Form} \tag{4.3}$$

Que puede leerse como: *si en el contexto* Γ *el término* A *es un tipo bien formado y, al extender dicho contexto con la hipótesis de que* $x : A$*, es posible inferir que también lo es* B*, entonces* $\Sigma\, x : A.\, B$ *es un tipo en* Γ. El equivalente en Maude de esta regla de formación, es la siguiente:

```
fmod LAMBDA-SYNTAX is
  ...
  op infer : Ctx Term -> [Term] .
  ceq infer(G, Sigma(V : A, B)) = Type
    if Type := norm(infer(G, A))
    /\ Type := norm(infer(G ; V : A, B)) .
endfm
```

Obsérvese que su interpretación no es muy distinta a lo que la regla *(2.27)* formaliza: podemos inferir que $Sigma(V, A, B)$ en la ecuación, es un tipo si se cumple que A es un tipo en Γ y, en el mismo contexto extendido, V es de tipo A, B es también un tipo bien formado.

La ejecución de este prototipo se convierte en la reducción del término a su forma canónica, y `reduce` evalúa el término hasta obtener el tipo inferido o bien, el tipo al que el término pueda habitar.

En definitiva, Maude provee una forma sumamente natural de implementar reglas de inferencia y la semántica de un lenguaje de manera ecuacional y visual los pasos en los que se computa cada expresión que se suministre en pasos de reducción.

Capítulo 5

Un lenguaje dependientemente tipado en Maude

El lenguaje que se propone en el presente trabajo, parte de un subconjunto del lambda cálculo simplemente tipado, según lo descrito por Löh et al. (2010) en su publicación sobre un lambda cálculo dependientemente tipado, donde nuestra implementación, pese a que difiere ligeramente de la misma, definirá las bases para el prototipo de lenguaje que se estará formalizando. Asimismo, se ha elegido Maude como lenguaje base, por su simpleza para formalizar reglas de inferencia, como se ha podido demostrar.

5.1. Decisiones de diseño y alcance

El prototipo que se describe en este capítulo y en capítulo 6 se restringe en ciertos puntos respecto de la teoría presentada en capítulo 2. Se enuncian aquí de forma conjunta, indicando en cada caso el motivo de la decisión y la consecuencia que estas puedan traer en el resultado final.

5.1.1. Sustitución que evita capturas

La operación `subst` implementada adopta el enfoque *naive*, es decir, se reemplazan las variables respectivamente sin verificar que la variable a reemplazar esté ligada por un cuantificador. La razón de esto, es que los términos que se manipulan en los casos de estudio son cerrados, por lo que la captura no se produce. También, la sustitución no renombra las variables ligadas con variables frescas. Extenderla a términos abiertos requeriría una operación auxiliar de renombrado con variables frescas que, por simpleza del prototipo, no se implementará.

5.1.2. Anotación explícita de términos

La gramática de apartado 5.2 contempla el constructor de anotación $e :: \rho$, pero el prototipo no lo incorpora como nodo independiente. La anotación es explícita y cada ecuación de Maude especifica el tipo que se espera de un término en concreto.

5.1.3. Universo único

A diferencia de lo propuesto por Li (2014), cuyo trabajo se sitúa en la teoría de tipos de Martin-Löf, donde los cocientes se incorporan como una extensión de la teoría y se distingue entre los universos *Type* y *Prop*, nuestro sistema se limita a un único universo $\star$, que a su vez habita en un *Sort* de Maude. En su formulación, toda relación $\sim$ habita en *Prop*, el universo de las proposiciones; en la nuestra, al no existir tal distinción, $\sim$ habita en $\star$. Esta decisión simplifica las reglas de formación, introducción, eliminación y cómputo, y sus consecuencias sobre el tratamiento de la relación se discuten en la regla de formación del cociente.

5.1.4. Regla de cómputo del cociente

No se declara una ecuación independiente para *Q-Comp*: su contenido operacional se realiza dentro de la normalización de `qelim`, que contrae el eliminador cuando su tercer argumento es una clase canónica.

5.2. Sintáxis

$$
\begin{aligned}
e, \rho, k := \quad & \alpha && (1)\\
\mid \quad & e :: \rho && (2)\\
\mid \quad & \star && (3)\\
\mid \quad & \forall x :: \rho . \rho' && (4)\\
\mid \quad & \lambda x.\, e && (5)\\
\mid \quad & e\ e' && (6)
\end{aligned}
$$

Nótese, como anteriormente se ha cubierto, los términos y los tipos comparten el mismo nivel semántico y no existe distinción entre ambos, por tanto, pudiese un término ocurrir en un tipo. El algebra inicial del que parte el prototipo del lenguaje, se compone del siguiente árbol de sintáxis abstracta:

```
fmod LAMBDA-QTT is
  ...
  sort Var .
  sort Term .
  subsort Var < Term .
  vars N N' : String .
  vars T T1 T1' T2 S A F B R QT Resp Q : Term .
  vars X X' V V' : Var .
  op var : String -> Var [ctor] .
  op Type : -> Term [ctor] .
  op Pi(_:_,_) : Var Term Term -> Term [ctor] .
  op λ_:_._ : Var Term Term -> Term [ctor prec 40] .
  op _·_ : Term Term -> Term [ctor prec 15 gather (E e)] .
  ...
endfm
```

El segmento de constructor de tipos es de mayor interés. Es tentador definir un subsort de Term, tal que, dado un *Type* también podamos obtener un *Term* de forma intercambiable:

```
subsort Type < Term .
```

Pero esto es incorrecto, pues no tiene *Type* un orden estructuralmente menor que *Term*, por el contrario, dado que defininimos un cálculo dependiente, términos y tipos vienen siendo lo mismo semánticamente.

5.3. Sustitución

Las reglas de sustitución para nuestro lenguaje no son necesariamente distintas de las del lambda cálculo sin tipos discutidas en un capítulo previo, aún así, existen diferencias que son notables en la sustitución del cálculo dependiente.

1. Las anotaciones de tipo de los binders para el tipo Π, es decir, A en $\lambda x : A.\, t$ y en $\Pi x : A.\, B$, son ahora términos y por tanto pueden contener ocurrencias libres de variables. La sustitución debe aplicarse de forma recursiva sobre el término.
2. Cuando se sustituye bajo un binder de variable y distinta de x, el cuerpo del binder se sustituye normalmente; cuando $y = x$, la sustitución se detiene en la anotación de tipo pero no avanza al cuerpo, pues x queda capturada por el binder.

La regla correspondiente a la sustitución en el tipo de función dependiente es omitida por Löh et al. (2010) sin embargo, es prudente mencionarla en nuestra implementación.

$$\Pi y : A.B[x := s] = \Pi y : A[x := s].B[x := s]$$

La regla es trivial para λ y Σ que describirémos posteriormente. Para el resto de casos, la sustitución se propaga estructuralmente.

A diferencia del enfoque empleado por Stehr and Meseguer (2004), el cálculo de sustituciones no es explícito, y cada regla es expresada de manera ecuacional. Dado que los términos del prototipo son cerrados y no hay variable que pueda ser capturada por un binder, una aproximación *naive* es suficiente para los casos de uso. Luego, la implementación en Maude define las reglas de cómputo para la sustitución en el cálculo dependientemente tipado, se define una regla por cada constructor del cálculo, definiendo la condición de igualdad o desigualdad sobre los nombres de las variables, expresadas en las ecuaciones:

```
fmod LAMBDA-QTT is
  ...
  op _[_ / _]: Term Var Term -> Term .

  eq X[X / S] = S .
  ceq X[X' / S] = X if X =/= X' .
  eq (T1 · T2)[X / S] = (T1[X / S]) ·(T2[X / S]) .
  eq Type[X / S] = Type .
  ceq (λ X : A . T)[X / S] = λX : A[X / S] . T
    if true = true .
  ceq (λ X : A . T)[X' / S] =
    λ X : A[X' / S] . T[X' / S]
    if X =/= X' .
  ceq Pi(X : A, B)[X / S] = Pi(X : A[X / S], B)
    if true = true .
  ceq Pi(X : A, B)[X' / S] =
    Pi(X : A[X' / S], B[X' / S])
```

```
18     if X =/= X' .
19   ...
20 endfm
```

5.4. Normalización

Se define, a continuación, la semántica operacional *big step* del proceso de normalización para cada término en el lenguaje, y con ella, la ecuación de Maude que corresponde a cada regla de evaluación.

$$\frac{}{X \Downarrow X}\ \text{Var} \qquad \texttt{eq norm(X) = X .}$$

$$\frac{}{\mathsf{Type} \Downarrow \mathsf{Type}}\ \text{Type} \qquad \texttt{eq norm(Type) = Type .}$$

$$\frac{A \Downarrow A' \quad B \Downarrow B'}{\text{Pi}(X : A,\, B) \Downarrow \text{Pi}(X : A',\, B')}\ \Pi \qquad \texttt{eq norm(Pi(X : A, B)) = Pi(X : norm(A), norm(B)) .}$$

$$\frac{A \Downarrow A' \quad T \Downarrow T'}{\lambda X : A.\, T \Downarrow \lambda X : A'.\, T'}\ \lambda \qquad \texttt{eq norm(}\lambda\texttt{ X : A . T) = }\lambda\texttt{ X : norm(A) . norm(T) .}$$

$$\frac{T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2'}{T_1 \cdot T_2 \Downarrow \text{normApp}(T_1', T_2')}\ \text{App} \qquad \frac{T[X := S] \Downarrow T'}{\text{normApp}(\lambda X : A.\, T,\, S) \Downarrow T'}\ \beta$$

$$\frac{}{\text{normApp}(T_1, T_2) \Downarrow T_1 \cdot T_2}\ \text{App-otherwise} \qquad \texttt{eq normApp(T1, T2) = (T1 · T2) [owise] .}$$

$$\frac{}{\text{Nat} \Downarrow \text{Nat}}\ \text{Nat} \qquad \texttt{eq norm(Nat) = Nat .}$$

$$\frac{}{\text{zero} \Downarrow \text{zero}}\ \text{zero} \qquad \texttt{eq norm(zero) = zero .}$$

$$\frac{T \Downarrow T'}{\text{suc}(T) \Downarrow \text{suc}(T')}\ \text{suc} \qquad \texttt{eq norm(suc(T)) = suc(norm(T)) .}$$

$$\frac{A \Downarrow A' \quad T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2' \quad T \Downarrow T'}{\text{natrec}(A, T_1, T_2, T) \Downarrow \text{normNr}(A', T_1', T_2', T')}\ \text{natrec}$$

$$\frac{}{\text{normNr}(A, T_1, T_2, \text{zero}) \Downarrow T_1}\ \text{normNr-zero}$$

$$\frac{(T_2 \cdot T) \cdot \text{normNr}(A, T_1, T_2, T) \Downarrow V}{\text{normNr}(A, T_1, T_2, \text{suc}(T)) \Downarrow V}\ \text{normNr-suc}$$

$$\frac{}{\text{normNr}(A, T_1, T_2, T) \Downarrow \text{natrec}(A, T_1, T_2, T)}\ \text{normNr-otherwise}$$

$$\frac{A \Downarrow A' \quad B \Downarrow B'}{\text{Sigma}(X, A, B) \Downarrow \text{Sigma}(X, A', B')}\ \Sigma$$

```
eq norm(Sigma(X, A, B)) = Sigma(X, norm(A), norm(B)) .
```

$$\frac{T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2' \quad A \Downarrow A'}{\text{pair}(T_1, T_2, A) \Downarrow \text{pair}(T_1', T_2', A')}\ \text{Pair}$$

```
eq norm(pair(T1, T2, A)) = pair(norm(T1), norm(T2), norm(A)) .
```

$$\frac{T \Downarrow T'}{\text{fst}(T) \Downarrow \text{normFst}(T')}\ \text{fst} \qquad \frac{}{\text{normFst}(\text{pair}(T_1, T_2, A)) \Downarrow T_1}\ \text{normFst}$$

$$\frac{}{\text{normFst}(T) \Downarrow \text{fst}(T)}\ \text{normFst-otherwise}$$

```
eq norm(fst(T)) = normFst(norm(T)) .
```

$$\frac{T \Downarrow T'}{\text{snd}(T) \Downarrow \text{normSnd}(T')}\ \text{snd} \qquad \frac{}{\text{normSnd}(\text{pair}(T_1, T_2, A)) \Downarrow T_2}\ \text{normSnd}$$

$$\frac{}{\text{normSnd}(T) \Downarrow \text{snd}(T)}\ \text{normSnd-otherwise}$$

```
eq norm(snd(T)) = normSnd(norm(T)) .
```

5.5. Verificación de tipos

La verficación de tipos que se implementa es bidireccional; se proponen las ecuaciones *check* ($\downarrow$) e $infer$ ($\uparrow$) para verificar que un término es de un tipo o respectivamente, sintetizar su tipo dado un contexto Γ cuando no hay suficiente información para ratificar su estructura en un principio. Para propósitos prácticos, cada término en el lenguaje que se formaliza tiene una anotación, es decir, está anotado con su signatura o el tipo en el que habita, y cualquier término no tipado será rechazado por el typechecker.

```
op check : Ctx Term Term -> [Bool] .        (5.1)
op infer : Ctx Term -> [Term] .             (5.2)
op tyEq : Term Term -> Bool .               (5.3)
```

$TyEq$ determina posteriormente si $T_1 =_\beta T_2$ luego de normalizar exhaustivamente, como consecuencia de la ocurrencia de términos en los tipos. La correspondencia entre las reglas de tipado y el código Maude está basada directamente en las del lambda cálculo dependientemente tipado sugeridas por Löh et al. (2010). Como observación, la implementación que se detalla diverge ligeramente en los constructores $Sigma$, fst y snd, pues no son propias del lambda dependientemente tipado que inicialmente Löh propone, pero jugarán un papel fundamental en los casos de uso que al momento de presentar los tipos cociente en nuestro sistema, se mostrarán.

$$\frac{}{\Gamma \vdash \star \uparrow \star}\ Type$$

```
eq infer(G, Type) = Type.
```

$$\frac{\Gamma \vdash A \downarrow \star \quad \Gamma, x : A \vdash B \uparrow \star}{\Gamma \vdash Pi(x, A, B) \uparrow \star}\ Pi$$

```
ceq infer(G, Pi(V, A, B)) =
Type if Type := norm(infer(G,
A)) /\ Type := norm(infer(G ;
V : A, B)) .
```

$$\frac{\Gamma \quad lookup(x) = \tau}{\Gamma \vdash x \uparrow \tau}\ Var$$

```
eq infer(G, var(N)) =
lookup(var(N), G).
```

$$\frac{\Gamma \vdash T_1 \uparrow Pi(x, A, B) \quad \Gamma \vdash T_2 \downarrow A}{\Gamma \vdash app(T_1, T_2) \uparrow B[x := T_2]}\ App$$

```
ceq infer(G, app(T1, T2))
= norm(subst(B, V, T2)) if
Pi(V, A, B) := norm(infer(G,
T1)) /\ true := check(G, T2,
A) .
```

$$\frac{\Gamma \vdash A \uparrow \star \quad \Gamma, x : A \vdash B \uparrow \star}{\Gamma \vdash \lambda x : A.T \downarrow Pi(x, A, B)}\ Lam$$

```
ceq infer(G, lam(V, A, T))
= Pi(V, A, B) if Type :=
norm(infer(G, A)) /\ B :=
infer(G ; V : A, T) .
```

$$\frac{}{\Gamma \vdash Nat \downarrow \star}\ Nat$$

```
eq infer(G, Nat) = Type.
```

$$\frac{}{\Gamma \vdash zero \downarrow Nat}\ zero$$

```
eq infer(G, zero) = Nat .
```

$$\frac{\Gamma \vdash T \downarrow Nat}{\Gamma \vdash suc(T) \uparrow Nat}\ suc$$

```
ceq infer(G, suc(T)) = Nat if
true := check(G, T, Nat).
```

$$\frac{\Gamma \vdash A \downarrow \star \quad \Gamma, x : A \vdash B \uparrow \star}{\Gamma \vdash Sigma(V, A, B) \uparrow \star}\ \Sigma$$

```
ceq infer(G, Sigma(V,
A, B)) = Type if Type :=
norm(infer(G, A)) / Type :=
norm(infer(G ; V : A, B)) .
```

$$\frac{\Gamma \vdash T_1 : A \quad \Gamma \vdash T_2 : B[x := T_1]}{\Gamma \vdash pair(T_1, T_2, Sigma(V, A, B)) \uparrow Sigma(V, A, B)}\ Pair$$

```
ceq infer(G, pair(T1, T2,
Sigma(V, A, B))) = Sigma(V,
A, B) if true := check(G,
T1, A) / true := check(G, T2,
norm(subst(B, V, T1))) .
```

$$\frac{\Gamma \vdash T \uparrow Sigma(V, A, B)}{\Gamma \vdash fst(T) \uparrow A}\ fst$$

```
ceq infer(G, fst(T)) =
A if Sigma(V, A, B) :=
norm(infer(G, T)) .
```

$$\frac{\Gamma \vdash T \uparrow Sigma(V, A, B)}{\Gamma \vdash snd(T) \uparrow B[x := fst(T)]}\ snd$$

```
ceq infer(G, fst(T)) =
A if Sigma(V, A, B) :=
norm(infer(G, T)) .
```

Capítulo 6

Extensión con tipos cociente

El lambda cálculo dependientemente tipado implementado en el capítulo anterior será la base sobre la que extenderémos el sistema con tipos cociente, guiándonos de lo sugerido por Li (2014). Como en el capítulo precedente, se listan las reglas de formación, introducción, eliminación y computación, y a cada una de ellas se le hace corresponder la ecuación Maude que la realiza en el prototipo.

Conviene precisar, antes de entrar en las reglas, en qué sentido una ecuación de Maude computa una regla de tipado, pues de esa correspondencia depende buena parte de lo que el prototipo permite afirmar. Una regla de inferencia define de forma inductiva una relación de derivabilidad. La conclusión $\Gamma \vdash t : A$, por ejemplo, se considera derivable cuando lo son sus premisas. Su traslado a la lógica de reescritura adopta un punto de vista computacional. La función `infer`, en lugar de comprobar una derivación ya construida, la produce por reducción sobre el álgebra de términos, y las condiciones de *matching* que acompañan a cada `ceq`, introducidas mediante el operador `:=`, desempeñan el papel de las premisas, de tal modo que la ecuación se dispara únicamente cuando cada una de ellas encaja con la forma normal del término que le corresponde. La derivabilidad del juicio se manifiesta entonces como la buena definición del resultado de `infer`, por tanto, un término bien tipado reduce al tipo que se le infiere, mientras que uno mal tipado no queda cubierto por ecuación alguna, y así, `infer` permanece sin reducir el kind [Term]. Esta es la lectura que adopta en nuestro sistema la equivalencia *Cómputo = Deducción* enunciada en capítulo 4, pues derivar el tipo de un término y normalizar la aplicación de `infer` sobre él constituyen el mismo cómputo.

La gramática presentada en el capítulo anterior se extiende con tres nuevos constructores, que se corresponden con la formación del cociente, la formación de una clase de equivalencia y la eliminación por *lifting*, respectivamente:

$$
\begin{aligned}
e, \rho, k := \quad & \ldots && (*)\\
\mid \quad & (A/R) && (7)\\
\mid \quad & [t : A/R] && (8)\\
\mid \quad & \mathrm{qelim}(f, resp, q) && (9)
\end{aligned}
$$

Donde (7) construye el tipo cociente $A/\sim$ a partir de un tipo A y de una relación R; (8) construye la clase de equivalencia $[t]$ de un representante $t : A$, anotada con el tipo cociente al que pertenece; y (9) eleva una función f definida sobre representantes a una función definida

sobre el cociente, exigiendo un testigo *resp* de que f respeta la relación. Su correspondencia en Maude, se indica como sigue:

```
op _/_ : Term Term -> Term [ctor prec 25] .
op [_:_] : Term Term -> Term [ctor] .
op qelim : Term Term Term -> Term [ctor] .
```

La sustitución se propaga sobre cada constructor y recursivamente sobre los tipos presentes en cada uno de ellos. Una vez más, no se presenta captura de variables porque ningún constructor introduce un binder sobre la variable que se sustituye:

```
eq (A / R)[V / S] = (A[V / S]) / (R[V / S]) .
eq [A : R][ V / S] = [A[V / S] : R[V / S]] .
eq qelim(F, A, R)[V / S] = qelim(F[V / S], A[V / S], R[V / S]) .
```

6.1. Introducción

$$\frac{\Gamma \vdash t : A}{\Gamma \vdash [t : A/R] : A/R}\ \text{Q-Intro} \tag{6.1}$$

Según lo fijado por la regla, la ecuación en Maude comprueba que el tipo de un representante $[t]$ en el cociente es de tipo $A/\sim$, si en principio es posible tener una derivación del hecho de que en el mismo contexto Γ, t habita en A.

```
ceq infer(G, [T : A / R]) = A / R
  if true := check(G, T, A) .
```

La decisión de anotar el tipo cociente sobre la clase responde al mismo principio que en el capítulo anterior abordamos con las tuplas del tipo Σ, donde pair posee su anotación. Sin dicha anotación, la síntesis del tipo de una clase sería ambigua, pues un mismo representante $t : A$ pertenece a tantos cocientes como relaciones puedan definirse sobre A. Portando la clase su tipo cociente, la inferencia permanece dirigida por la sintaxis y, por tanto, se mantiene decidible.

Puesto que A y R son dentro de nuestro universo $\star$, términos, es sensato normalizar ambos argumentos. De manera que la ecuación en Maude es congruente:

```
eq norm(A / R) = norm(A) / norm(R) .
```

6.2. Formación

$$\frac{\Gamma \vdash A \uparrow \star, \quad \Gamma a, b : A \vdash x \sim y \downarrow Pi(a : A,\ Pi(b : A,\ Type))}{\Gamma \vdash A/\sim \quad \uparrow \star}\ \text{Q-Form} \tag{6.2}$$

La regla de formación sostiene que para inferir que $A/\sim$ es de tipo $\star$, primero debe atestiguar que la relación de equivalencia entre a y b es transitiva, reflexiva y simétrica y, además, habitar en el tipo Π. Adicionalmente, a y b habitan A.

```
1  ceq infer(G, A / R) = Type
2  if Type := norm(infer(G, A))
3  /\ true := check(G, R, Pi(a : A, Pi(b : A, Type))).
```

En esta regla se toma la decisión de emplear un único universo. En el sistema de Li (2014) la relación $\sim$ habita en $Prop$, el universo de las proposiciones, cuya característica es la irrelevancia de pruebas, donde dada una proposición $a : Prop$ y dos testigos cualesquiera $h1 : a$, $h2 : a$, ambos $h1$ y $h2$ se consideran indistinguibles para el sistema de tipos (The mathlib Community, 2026). Al disponer nuestro sistema de un solo universo $\star$, la premisa que `infer` exige de R es únicamente que habite en $\Pi\, a : A.\, \Pi\, b : A.\, \mathsf{Type}$, esto es, que sea una familia de tipos sobre A. El sistema admite en ese lugar cualquier familia de esa signatura, y en consecuencia no distingue una relación de equivalencia de una familia arbitraria de tipos, ni verifica que R sea reflexiva, simétrica ni transitiva. Estas propiedades se asumen como hipótesis, como en el modelo de setoides, en el que la relación se transporta como parámetro del contexto y su carácter de equivalencia queda a cargo de quien la suministra. Su comprobación efectiva requeriría, además, la capacidad de construir y transportar pruebas de igualdad empleando el eliminador J del tipo identidad, como se discute más adelante el sistema no posee.

6.3. Eliminación y Computación

$$\frac{\Gamma \vdash Q \uparrow A/\sim \quad F \uparrow Pi(a : A, B)}{\Gamma \vdash qelim(F, Resp, Q) \uparrow B}\ \text{Q-Elim} \tag{6.3}$$

Tanto la eliminación como la computación, juegan un papel importante en esta extensión a los cocientes, ellos indican a Maude la forma en que deben reducirse las reglas del lenguaje. En esta regla en concreto, inferimos que *qelim* es de tipo B, asumiendo que F es una función cualquiera donde su *Resp* es válida. Nótese que en nuestra implementación de Maude, agregamos como clausula la inferencia de que Q es de tipo $A/\sim$, es decir, A es un testigo bien definido bajo el cociente en Γ y es válido aplicar F sobre el.

```
1  ceq infer(G, qelim(F, Resp, Q)) = B
2  if A / R := norm(infer(G, Q))
3    /\ Pi(V : A , B) := norm(infer(G, F))
4    /\ true := check(G, Resp,
5        Pi(a : A , Pi(b : A ,
6            Pi(p : (R ·a) · b ,
7            Id(B, F ·a, F · b))))) .
```

La regla de computación describe cómo se comporta *lifting* en una función al aplicarse a una clase canónica. Nótese que $Q - Comp$ y $Q - Elim$ van de la mano, como se ha mencionado, y la razon es que la eliminación en principio describe las reglas para poder aplicar una función a un miembro del cociente y, por otro lado, la regla de cómputo describe, cómo de verdad obtener un valor a partir de ella. Nuestra implementación no tiene una regla concreta de *qcomp*, por lo que realmente basta y es mucho más práctico para nuestro propósito, cubrir directamente *qelim*

tomando en cuenta lo indicado por ambas reglas. Siguiendo pues, lo sugerido por Li (2014), al aplicar $\hat{f}$ a la clase $[a]$ de un representante a, se obtiene $f(a)$:

$$\frac{\mathrm{app}(F, T) \Downarrow V}{\mathrm{qelim}(F, A, [T : QT]) \Downarrow V}\ \text{qelim-}\beta$$

$$\frac{F \Downarrow F' \quad A \Downarrow A' \quad R \Downarrow R'}{\mathrm{qelim}(F, A, R) \Downarrow \mathrm{qelim}(F', A', R')}\ \text{qelim-otherwise}$$

En el prototipo de Maude, esta igualdad se realiza directamente como una regla de reescritura orientada de izquierda a derecha. Cuando el tercer argumento del eliminador es una clase canónica $\mathrm{qclass}(T, QT)$, el *lifting* se contrae aplicando F al representante T:

```
1  eq norm(qelim(F, A, R)) = normQe(norm(F), norm(A), norm(R)) .
2  eq normQe(F, A, [T : QT]) = norm(F ·T) .
3  eq normQe(F, A, R) = qelim(F, A, R) [owise] .
```

La primera ecuación traslada la normalización de `qelim` a la función auxiliar `normQe`, previa normalización de sus tres argumentos. Es `normQe` quien decide, una vez conocida la forma normal de esos argumentos, si el eliminador puede contraerse. La segunda ecuación es la contraparte operacional de *Q-Comp*: cuando el tercer argumento es una clase canónica $[T : QT]$, el *lifting* se reduce a la aplicación $F \cdot T$, que a su vez se normaliza mediante la β-reducción ya definida para el lenguaje base, obteniéndose $f(t)$ a partir de $\hat{f}([t])$. La tercera ecuación, marcada con `owise`, cubre el caso en que ese tercer argumento no es una clase canónica, por ejemplo una variable. Al no disponer de información suficiente sobre qué es ese argumento, no hay redex que contraer, de modo que los tres componentes quedan ya normalizados por la primera ecuación y el eliminador permanece sin reducir.

A modo de ilustración, considérese el cociente de Nat por una relación R cualquiera y la elevación de la función sucesor. Tomando como función $\lambda x : \mathsf{Nat}.\, \mathrm{suc}(x)$ y como elemento la clase del representante *zero*, la síntesis del tipo y la computación proceden como sigue:

```
1 reduce in calc : norm(qelim(lam(var("x"), Nat, suc(var("x"))), var("resp"), qclass(zero, qtype(Nat, var("R
      "))))) .
2 rewrites: 12 in 0ms cpu (0ms real) (~ rewrites/second)
3 result Term: suc(zero)
4 Maude>
```

En este ejemplo, dado que tenémos una clase canónica $[zero]$ donde R sí respeta la equivalencia, el tercer argumento de *qelim* tiene la forma *(T, R)*, es decir, un tipo bien formado y el cociente en el que está definida su relacíon de equivalencia, por tanto, la ecuación se reduce desenvolviendo la clase de equivalencia $[zero]$, aplicando f a su representante, en este caso *zero*, donde trivialmente obtenemos *suc(zero)* como resultado.

Conviene detenerse para discutir el alcance de la igualdad proposicional en nuestro lenguaje. El sistema implementa el tipo identidad *Id*, su constructor de reflexividad *refl*, y la regla $Q - Ax$, que se muestra:

$$\frac{\Gamma \vdash a, b : A \quad \Gamma \vdash p : a \sim b}{\Gamma \vdash Qax(p) : [a] =_{A/\sim} [b]}\ \text{Q-Ax} \tag{6.4}$$

Dada una prueba p de que dos representates a y b están relacionados, $Q - Ax$ produce una prueba p que enuncia que sus representantes son iguales también dentro del cociente. El sistema puede, por tanto, declarar de manera interna la igualdad proposicional $[a] =_{A/\sim} [b]$, y no asumirla en la metateoría.

Amerita ubicar esta regla frente a la manera en que los probadores de teoremas basados en teoría de tipos incorporan este mismo principio. En Lean, la propiedad correspondiente, `Quot.sound`, se introduce como un axioma en el kernel, esto es, como una constante postulada que afirma la igualdad de las clases de dos representantes relacionados sin que le corresponda regla de cómputo alguna; en Coq, por otro lado, cuando el cociente se añade por vía axiomática, la situación es la misma. En nuestro sistema, Q-Ax se expresa como una regla de tipado que `infer` realiza mediante una ecuación condicionada, en lugar de asumirse de forma externa en la teoría. La igualdad $[a] =_{A/\sim} [b]$ queda así producida por el propio sistema de tipos a partir de una prueba de que $a \sim b$, y su estatus es el de un juicio derivable en el cálculo. Esta discrepancia entre postular tal propiedad del cociente como un axioma y especificarla como regla es donde la lógica de reescritura resulta de interés en nuestra aproximación, pues la premisa $\Gamma \vdash p : a \sim b$ se computa, y comprueba por reducción, luego la conclusión se obtiene como forma normal de `infer` sobre el término $Qax(p)$.

Adicionalmente, como se ha detallado con anterioridad, el lenguaje no implementa el eliminador de la identidad J, por lo que las pruebas de igualdad pueden enunciarse, pero no pueden emplearse para transportar propiedades de un lado de la igualdad al otro. La igualdad proposicional es, en este sentido, de solo introducción.

Esta ausencia tiene una consecuencia sobre el testigo *Resp* del eliminador del cociente. Su tipo es

$$Resp : (a, b : A) \to a \sim b \to Id(B, f(a), f(b)),$$

esto es, la prueba de que f respeta la relación. Maude verifica que el *Resp* suministrado tenga efectivamente ese tipo, pero sin J no dispone de los medios para construir tales pruebas para funciones no triviales, pues ello requiere la congruencia que J habilita. En consecuencia, *Resp* se suministra como hipótesis en el contexto, del mismo modo que la relación $\sim$ se asume como una equivalencia sin que el sistema verifique su reflexividad, simetría y transitividad. Estas asunciones no comprometen la simulación de los cocientes en los casos de uso presentados posteriormente.

Con las reglas de formación, introducción, eliminación y computación ya implementadas, el lenguaje dispone de tipos cociente en el sentido intensional de Li (2014). Los constructores fst, snd y el tipo Σ que se incorporaron en el capítulo anterior cobran ahora utilidad, pues permiten expresar los casos de uso característicos del cociente, como los enteros representados por pares de naturales o los multiconjuntos, ejemplos que se desarrollan en capítulo 7.

Capítulo 7

Casos de uso y validación

7.1. Los enteros como pares de naturales

La representación canónica de la aplicación de los cocientes viene dada por los enteros como pares de naturales $(\mathbb{N} \times \mathbb{N})/\mathbb{R}$. Es posible demostrar la factibilidad del prototipo mediante los casos de uso que se presentan a continuación.

De acuerdo a la regla $Q - Form$, podemos afirmar que dado un contexto Γ, y una relación $R : (a : Nat \times Nat) \to (b : Nat \times Nat) \to Type$, es posible inferir que su tipo es $Type$.

```
red infer(
  mt ; var("R") : Pi(var("a"), Sigma(var("_"), Nat, Nat), Pi(var("b"), Sigma(var("_"), Nat, Nat), Type)),
  qtype(Sigma(var("_"), Nat, Nat), var("R"))
) .

result Term: Type
```

En este primer caso se declara en el contexto una relación R sobre el tipo $Nat \times Nat$, y se pregunta al sistema por el tipo del cociente que dicha relación induce. Maude resuelve la ecuación asociada a $Q - Form$ verificando sus dos premisas: que $Nat \times Nat$ es un tipo bien formado y que R tiene la signatura de una relación binaria sobre ese tipo, es decir, que habita en $\Pi a : (Nat \times Nat).\, \Pi b : (Nat \times Nat).\, Type$. Satisfechas ambas por reducción, el sistema concluye que el cociente es de tipo $Type$, tal y como se observa en el resultado.

Una vez formado el cociente, la regla $Q - Intro$ permite construir sus clases. El siguiente caso comprueba que un representante concreto, el par $(0, 1)$, habita efectivamente en el cociente.

```
red infer(
  mt ; var("R") : Pi(var("a"), Sigma(var("_"), Nat, Nat), Pi(var("b"), Sigma(var("_"), Nat, Nat), Type)),
  qclass(pair(zero, suc(zero), Sigma(var("_"), Nat, Nat)), qtype(Sigma(var("_"), Nat, Nat), var("R")))
) .

result Term: qtype(Sigma(var("_"), Nat, Nat), var("R"))
```

Obtenemos como tipo el propio cociente, porque Maude comprueba, mediante la premisa de $Q - Intro$, que el par $(0, 1)$ habita en $Nat \times Nat$ y, en consecuencia, que su clase $[(0, 1)]$ habita en $(Nat \times Nat)/R$. La anotación del tipo cociente sobre la clase es lo que permite que esta síntesis permanezca dirigida por la sintaxis, según se justificó al presentar la regla.

7.2. Elevación de funciones sobre el cociente

El eliminador `qelim` permite elevar una función definida sobre representantes a una función definida sobre el cociente. Antes de estudiar la consistencia de estas funciones, conviene comprobar que el eliminador no solo computa un valor, sino que además está bien tipado. El siguiente caso eleva la función sucesor sobre un cociente de Nat, computa su valor y sintetiza su tipo.

```
red norm(qelim(λ x : Nat . suc(x), resp, [zero : Nat / ~])) .

result Term: suc(zero)

red infer(mt, qelim(λ x : Nat . suc(x), resp, [zero : Nat / ~])) .

result Term: Nat
```

La primera reducción corresponde a la regla de cómputo: al encontrar una clase canónica $[zero]$ como tercer argumento, Maude desenvuelve la clase y aplica la función a su representante, obteniendo $suc(zero)$. La segunda ejercita la regla $Q - Elim$ de tipado, donde el sistema infiere que $zero$ habita el cociente y que la función va de Nat a Nat, concluyendo que la elevación es de tipo Nat. Con ello queda comprobado que la operación no es solo computacionalmente correcta, sino también un término bien formado.

De la misma manera es posible elevar operaciones sobre los pares. Tomando la función que incrementa la primera componente, elevamos el sucesor de enteros y lo aplicamos al representante $(3, 1)$, que denota el entero $+2$.

```
red norm(qelim(
  λ x : Sigma(_, Nat, Nat) .
    [pair(suc(fst(x)), snd(x), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~],
  resp,
  [pair(suc(suc(suc(zero))), suc(zero), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
)) .

result Term: [pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
```

El resultado es la clase de $(4, 1)$, que representa el entero 3. Maude aplica la función al representante, incrementa la primera componente del par y devuelve la clase resultante.

7.3. Consistencia sobre representantes

Los pares $(3, 0)$, $(4, 1)$ y $(5, 2)$ son representantes distintos del entero 3, pues $3-0 = 4-1 = 5-2 = 3$. Al aplicar la función de negación $\text{negate} = \lambda(p : \mathbb{N} \times \mathbb{N}).\ [(snd(p),\ fst(p))]$ a cada uno mediante el eliminador del cociente `qelim`, esperamos que los resultados sean representantes del mismo entero, es decir, -3.

```
red norm(qelim(
  λ p : Sigma(_, Nat, Nat) .
    [pair(snd(p), fst(p), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~],
  resp,
  [pair(suc(suc(suc(zero))), zero, Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
)) .
result Term: [pair(zero, suc(suc(suc(zero))), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
```

```
red norm(qelim(
  λ p : Sigma(_, Nat, Nat) .
    [pair(snd(p), fst(p), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~],
  resp,
  [pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
)) .
result Term: [pair(suc(zero), suc(suc(suc(suc(zero)))), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
```

```
red norm(qelim(
  λ p : Sigma(_, Nat, Nat) .
    [pair(snd(p), fst(p), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~],
  resp,
  [pair(suc(suc(suc(suc(suc(zero))))), suc(suc(zero)), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~]
)) .
result Term: [pair(suc(suc(zero)), suc(suc(suc(suc(suc(zero))))), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat)
      / ~]
```

Los resultados obtenidos son $(0, 3)$, $(1, 4)$ y $(2, 5)$, respectivamente. El sistema devuelve la clase y, para abreviar nos referimos a su representante. Aunque son pares sintácticamente distintos, los tres representan el entero -3, pues $0 - 3 = 1 - 4 = 2 - 5 = -3$. Esto evidencia que la función `negate`, al respetar la relación de equivalencia R, produce resultados consistentes independientemente del representante elegido.

En contraste, una función como fst aplicada directamente a los pares produce resultados inconsistentes:

```
red norm(fst(pair(suc(suc(suc(zero))), zero, Sigma(_, Nat, Nat)))) .

red norm(fst(pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)))) .

red norm(fst(pair(suc(suc(suc(suc(suc(zero))))), suc(suc(zero)), Sigma(_, Nat, Nat)))) .
```

Se obtiene $fst((3, 0)) = 3$, $fst((4, 1)) = 4$ y $fst((5, 2)) = 5$, tres resultados distintos para el mismo entero $+3$. La función fst no respeta la relación de equivalencia y, por tanto, no podría proveer una prueba de respeto válida para `qelim`. El sistema de tipos cociente previene precisamente este tipo de inconsistencia al exigir dicha prueba.

7.4. Igualdad proposicional entre representantes

Los casos anteriores muestran que las operaciones que respetan la relación producen resultados consistentes, pero esa consistencia se observa por inspección de los representantes. Con el tipo identidad y la regla $Q - Ax$, el sistema puede además enunciar internamente que dos representantes relacionados dan la misma clase. Antes de llegar a ello, el siguiente caso comprueba el constructor más básico del tipo identidad sobre los naturales.

```
red infer(mt, refl(zero)) .

result Term: Id(Nat, zero, zero)
```

Maude infiere que $refl(zero)$ es un testigo de $Id(Nat, zero, zero)$, es decir, la prueba de que $zero$ es igual a sí mismo. La regla de la reflexividad sintetiza el tipo del punto y construye la identidad sobre él.

El caso central de esta sección aplica $Q - Ax$. Partiendo de un contexto donde R es una relación sobre los pares y p es una prueba de que los representantes $(3, 0)$ y $(4, 1)$ están relacionados, se pregunta al sistema por el tipo de qax.

```
red infer(
  mt ; var("R") : Pi(a : Sigma(_, Nat, Nat), Pi(b : Sigma(_, Nat, Nat), Type))
     ; var("p") : (var("R") ·pair(suc(suc(suc(zero))), zero, Sigma(_, Nat, Nat)))
                            · pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)),
  qax(pair(suc(suc(suc(zero))), zero, Sigma(_, Nat, Nat)),
      pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)),
      Sigma(_, Nat, Nat) / ~,
      var("p"))
) .

result Term: Id(Sigma(_, Nat, Nat) / ~,
               [pair(suc(suc(suc(zero))), zero, Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~],
               [pair(suc(suc(suc(suc(zero)))), suc(zero), Sigma(_, Nat, Nat)) : Sigma(_, Nat, Nat) / ~])
```

El resultado es la identidad $[(3, 0)] = [(4, 1)]$ en el cociente. Maude verifica, por la premisa de $Q - Ax$, que p tiene tipo $R\ (3, 0)\ (4, 1)$, esto es, que atestigua la relación entre ambos representantes, y a partir de ello sintetiza el tipo identidad entre sus clases. El propio tipo resultante exhibe los dos representantes dentro de sus respectivas clases. Conviene notar que estas siguen siendo formas normales sintácticamente distintas, por lo que el sistema no las identifica de manera definicional. Lo que $Q - Ax$ produce es la igualdad proposicional, un testigo de que ambas clases coinciden dentro del cociente, que era precisamente lo que en el caso de la negación se observaba solo de forma externa.

Por último, el tipo identidad, donde el siguiente caso construye la reflexividad sobre un término que primero debe reducirse.

```
red infer(mt, refl(norm((λ x : Nat . suc(x)) ·zero))) .

result Term: Id(Nat, suc(zero), suc(zero))
```

El argumento de $(\lambda x : Nat.\ suc(x))\ zero$ se normaliza a $suc(zero)$ antes de que $refl$ actúe, de modo que la identidad se construye sobre el resultado ya reducido. Esto muestra que la igualdad proposicional del sistema es coherente con la reducción del lenguaje base, pues opera sobre la forma normal de los términos y no sobre su forma sintáctica.

7.5. Multiconjuntos bajo relación de permutación

El próximo caso de uso son los multiconjuntos, obtenidos al cocientar las listas por la relación de permutación. Decimos que dos listas son equivalentes cuando una es permutación de la otra. Se toman como representantes $l_1 = [1, 2, 1]$ y $l_2 = [2, 1, 1]$, que denotan el mismo multiconjunto $\{1, 1, 2\}$.

Como listas, l_1 y l_2 son términos distintos, y el sistema, de manera correcta, los separa por comparación de sus formas normales, de modo que $\texttt{tyEq}(l_1, l_2)$ reduce a *false*. La regla Q-Ax produce un término de tipo $Id(List/\sim, [l_1], [l_2])$, que atestigua la igualdad de sus clases dentro del cociente.

```
red infer(G, qax(l1, l2, List / R, p)) .
result Term: Id(List / R, [l1 : List / R], [l2 : List / R])
```

Al igual que en los enteros, una función que respeta la permutación, como *length*, produce el mismo resultado sobre ambos representantes, mientras que una que no la respeta, como *head*, diverge.

Capítulo 8

Conclusiones, limitaciones y trabajo futuro

8.1. Conclusiones

El presente trabajo ha diseñado e implementado en Maude un lenguaje de programación dependientemente tipado y lo ha extendido con tipos cociente, cumpliendo el objetivo general planteado en la introducción. Partiendo de un núcleo basado en el cálculo lambda dependiente de Löh et al. (2010), se construyó un sistema con tipos Π, Σ, naturales y su eliminador, sobre el que se añadieron los constructores de formación, introducción, eliminación y computación del cociente siguiendo la formulación de Li (2014), junto con el tipo identidad.

La aportación central es haber mostrado que la lógica de reescritura constituye de una base sólida para el estudio de sistemas de tipos. Cada regla de inferencia del sistema de tipos se traslada de forma directa a una ecuación condicional de Maude, donde las premisas de la regla se corresponden con los fragmentos de la condición y la conclusión con el lado derecho. Esta correspondencia, que ya fue visible a lo largo de los capítulos 5 y 6, permite que la especificación del lenguaje sea a la vez su implementación ejecutable. La normalización de términos se realiza por reescritura ecuacional, y la verificación bidireccional de tipos reduce el chequeo a la comparación de formas normales.

Los casos de uso del capítulo 7 validan la factibilidad del prototipo. La construcción de los enteros como pares de naturales cocientados muestra que las funciones que respetan la relación de equivalencia, como la negación y el sucesor, producen resultados consistentes con independencia del representante elegido, mientras que una función que no la respeta, como lo es *head*, produce resultados divergentes. Con la incorporación del tipo identidad y de la regla $Q - Ax$, el sistema es capaz además de enunciar internamente que dos representantes relacionados dan la misma clase, obteniendo un testigo de esa igualdad proposicional dentro del cociente.

Asimismo, la formalización de los multiconjuntos como listas cocientadas bajo la relación de permutación complementa sustancialmente esta validación al introducir tipos inductivos recursivos. Este segundo caso de uso ilustra con precisión el contraste entre la igualdad del tipo base y la del cociente, dado que por un lado, el sistema separa sintácticamente términos distintos por normalización ecuacional, y por otro, la regla $Q - Ax$ permite construir un testigo en el tipo identidad que atestigua su igualdad proposicional dentro de la clase de equivalencia. De este modo, el ejemplo sirve como un valioso marco experimental para constatar de manera

práctica los límites del sistema ante la ausencia del eliminador J, mostrando cómo la reescritura ecuacional ayuda a diagnosticar y trazar la frontera operativa entre cocientes definicionales y proposicionales en la teoría de tipos intensional.

8.2. Limitaciones

El sistema habita en un único universo, con la regla $Type : Type$. Esta decisión simplifica la implementación al eliminar la jerarquía de universos, a costa de la consistencia lógica, pues admite la paradoja de Girard. El prototipo es, por tanto, un verificador de tipos y no un asistente de pruebas. Asimismo, su propósito es explorar (o bien, simular en cierto sentido) la mecánica de los cocientes.

La sustitución no renombra las variables ligadas con variables frescas, de modo que no evita la captura en el caso general. Los términos manipulados en los casos de uso son cerrados, por lo que la captura no llega a producirse, una mejora requeriría una operación auxiliar de renombrado y una mecánica más compleja de sustitucion explícita. En la misma línea, la igualdad de tipos se decide por comparación sintáctica de formas normales, lo que no contempla la α-equivalencia; términos que difieren solo en el nombre de sus variables ligadas se considerarían distintos.

El testigo *Resp* del eliminador del cociente se verifica a nivel de tipos, y no tiene efecto alguno sobre la reducción. El sistema computa la elevación de una función bajo la hipótesis de que dicha función respeta la relación de equivalencia, hipótesis que en el cómputo se toma como dada. Además, el sistema verifica que la relación suministrada habite en el tipo $\Pi\, a : A.\, \Pi\, b : A.\, \mathsf{Type}$, pero no que sea efectivamente una equivalencia, es decir, reflexiva, simétrica y transitiva.

Finalmente, aunque el lenguaje incorpora el tipo identidad, la regla de reflexividad y $Q - Ax$, no implementa el eliminador J del tipo identidad. En consecuencia, las pruebas de igualdad que el sistema construye son inertes, pueden enunciarse y exhibirse, pero no pueden emplearse para transportar propiedades de un lado de la igualdad al otro. La igualdad proposicional del prototipo es, en este sentido, de solo introducción.

8.3. Trabajo futuro

Una posible extensión, de la que podría beneficiarse la implementación es la codificación del eliminador J del tipo identidad, con su regla de cómputo sobre $refl$. Al ser J el principio del que se derivan las operaciones fundamentales sobre igualdades, su incorporación permite la operación de transporte de propiedades a lo largo de una igualdad, la congruencia de funciones y la simetría y transitividad de la identidad. Con ello, sería posible enunciar que dado dos representates $[a]$ y $[b]$ y un testigo $p : [a]\ [b]$ en el cociente, una propiedad P que es cierta para $[a]$ también lo sería para $[b]$.

Adicional a ello, una vez disponible J, sería posible construir un *Resp* que sea válido para funciones concretas, pues justo ahora el sistema no posee los medios para construir y, de esta forma, no asumirlo como hipótesis en el contexto.

Del mismo modo, la incorporación del constructor Let para definiciones locales dotaría al lenguaje de una mayor expresividad. A diferencia de los sistemas simplemente tipados, en la teoría de tipos dependientes let $x = t$ in u no puede reducirse a una aplicación lambda $(\lambda x : A.u)t$, ya que esta última trata a x como una variable abstracta dentro u. Un binder Let formal introduce una definición local en el contexto, permitiendo usar la igualdad definicional $x \equiv t$

durante la verificación de u. Asimismo, el uso de anotaciones explícitas en el prototipo podría mitigarse mediante un algoritmo de inferencia bidireccional más elaborado o el uso de unificador de términos, teniendo en cuenta que la reconstrucción total de tipos en sistemas dependientes compromete la decidibilidad del typechecker.

Bibliografía

Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.

Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, revised edition, 1984.

Henk P. Barendregt. An introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991. doi: 10.1017/S0956796800020025.

Henk P. Barendregt, Wil Dekkers, and Richard Statman. *Lambda Calculus with Types*. Perspectives in Logic. Cambridge University Press, 2013.

Gilles Barthe and Thierry Coquand. An introduction to dependent type theory. In Gilles Barthe, Peter Dybjer, Luís Pinto, and João Saraiva, editors, *Applied Semantics*, volume 2395 of *Lecture Notes in Computer Science*, pages 1–41. Springer, 2002. doi: 10.1007/3-540-45699-6_1.

Frédéric Blanqui. Type Safety of Rewrite Rules in Dependent Types. In Zena M. Ariola, editor, *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020)*, volume 167 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 13:1–13:14, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-155-9. doi: 10.4230/LIPIcs.FSCD.2020.13. URL `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.13`.

Frédéric Blanqui. *Type theory and rewriting*. Phd thesis, Université Paris XI, 2001. URL `https://arxiv.org/abs/cs/0610068`.

Manuel Clavel, Francisco Durán, Steven Eker, José Meseguer, Patrick Lincoln, Narciso Martí-Oliet, and Carolyn Talcott. *All About Maude: A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.

Robert L. Constable, Stuart F. Allen, H. M. Bromley, W. R. Cleaveland, J. F. Cremer, Robert W. Harper, Douglas J. Howe, T. B. Knoblock, N. P. Mendler, P. Panangaden, James T. Sasaki, and Scott F. Smith. *Implementing Mathematics with the Nuprl Proof Development System*. Prentice-Hall, 1986.

Thierry Coquand. Type theory. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Fall 2022 edition, 2022. URL `https://plato.stanford.edu/archives/fall2022/entries/type-theory/`.

Francisco Durán, Steven Eker, Santiago Escobar, Narciso Martí-Oliet, José Meseguer, Rubén Rubio, and Carolyn Talcott. Programming and symbolic computation in Maude. *Journal of Logical and Algebraic Methods in Programming*, 110:100497, 2020. doi: 10.1016/j.jlamp.2019.100497.

Martin Hofmann. *Extensional Concepts in Intensional Type Theory.* Phd thesis, 1995.

Martin Hofmann. *Syntax and Semantics of Dependent Types.* Cambridge University Press, 2009.

Nuo Li. *Quotient Types in Type Theory.* Phd thesis, 2014.

Andres Löh, Conor McBride, and Wouter Swierstra. A tutorial implementation of a dependently typed lambda calculus. *Fundamenta Informaticae*, 102(2):177–207, 2010.

María Manzano and Victor Aranda. Many-sorted logic, 2022. URL `https://plato.stanford.edu/entries/logic-many-sorted/`.

Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H. E. Rose and J. C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73–118. Elsevier, 1975. doi: 10.1016/S0049-237X(08)71945-1.

Samuel Mimram. *Program = Proof.* Independently published, 2020.

Rob Nederpelt and Herman Geuvers. *Type Theory and Formal Proof: An Introduction.* Cambridge University Press, 2014.

Aleksey Nogin. Quotient types: A modular approach. In Victor A. Carreño, César A. Muñoz, and Sofiène Tahar, editors, *Theorem Proving in Higher Order Logics*, Lecture Notes in Computer Science, pages 263–280. Springer, 2002.

Benjamin C. Pierce. *Types and Programming Languages.* MIT Press, 2002.

John C. Reynolds. The meaning of types: From intrinsic to extrinsic semantics. BRICS Report Series RS-00-32, BRICS, Department of Computer Science, University of Aarhus, 12 2000. URL `https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/typemeaning.pdf`.

Mark-Oliver Stehr and José Meseguer. Pure type systems in rewriting logic: Specifying typed higher-order languages in a first-order logical framework. In Olaf Owe, Stein Krogdahl, and Tom Lyche, editors, *From Object-Orientation to Formal Methods: Essays in Memory of Ole-Johan Dahl*, Lecture Notes in Computer Science, pages 334–375. Springer, 2004. doi: 10.1007/978-3-540-39993-3_16.

The Lean Language Reference Contributors. The lean language reference: Quotients, 2025. URL `https://lean-lang.org/doc/reference/latest/The-Type-System/Quotients/`.

The mathlib Community. `proof_irrel` — `Init.Core`, mathlib4 documentation, 2026. URL `https://leanprover-community.github.io/mathlib4_docs/Init/Core.html#proof_irrel`.

Apéndice A

Código Maude

```
fmod calc is
  pr STRING .
  sort Var .
  sort Term .
  subsort Var < Term .

  vars N N' : String .
  vars T T1 T1' T2 S A F B R QT Resp Q : Term .
  vars X X' V V' : Var .

  op x : -> Var [ctor] .
  op y : -> Var [ctor] .
  op f : -> Var [ctor] .
  op a : -> Var [ctor] .
  op b : -> Var [ctor] .
  op _ : -> Var [ctor] .
  op p : -> Var [ctor] .
  op resp : -> Var [ctor] .
  op ~ : -> Var [ctor] .

  op Nat : -> Term [ctor] .
  op zero : -> Term [ctor] .
  op suc : Term -> Term [ctor] .
  op natrec : Term Term Term Term -> Term [ctor] .
  op Sigma : Var Term Term -> Term [ctor] .
  op pair : Term Term Term -> Term [ctor] .
  op fst : Term -> Term [ctor] .
  op snd : Term -> Term [ctor] .
  op List : -> Term [ctor] .
  op nil : -> Term [ctor] .
  op cons : Term Term -> Term [ctor] .

  op var : String -> Var [ctor] .
  op Type : -> Term [ctor] .
  op Pi(_:_,_) : Var Term Term -> Term [ctor] .
  op λ_:_._ : Var Term Term -> Term [ctor prec 40] .
  op _·_ : Term Term -> Term [ctor prec 15 gather (E e)] .

  op refl : Term -> Term [ctor] .
  op Id : Term Term Term -> Term [ctor] .
  op qax : Term Term Term Term -> Term [ctor] .

  op _/_ : Term Term -> Term [ctor prec 25] . --- a / R, lease a modulo R
  op [_:_] : Term Term -> Term [ctor] . --- representante [t] en Q
  op qelim : Term Term Term -> Term [ctor] .

  op _[_ / _]: Term Var Term -> Term . --- substitutions, lease M[x / N] reemplazar x por N en el cuerpo
       de M

```

```
eq X[X / S] = S .
ceq X[X' / S] = X if X =/= X' .
eq (T1 · T2)[X / S] = (T1[X / S]) ·(T2[X / S]) .
eq Type[X / S] = Type .
ceq (λ X : A . T)[X / S] = λX : A[X / S] . T
  if true = true .
ceq (λ X : A . T)[X' / S] =
  λ X : A[X' / S] . T[X' / S]
  if X =/= X' .
ceq Pi(X : A, B)[X / S] = Pi(X : A[X / S], B)
  if true = true .
ceq Pi(X : A, B)[X' / S] =
  Pi(X : A[X' / S], B[X' / S])
  if X =/= X' .
eq Nat[X / S] = Nat .
eq zero[X / S] = zero .
eq suc(T)[X / S] = suc(T[X / S]) .
eq natrec(T1, T2, A, B)[X / S] =
    natrec(T1[X / S], T2[X / S], A[X / S], B[X / S]) .
ceq Sigma(X, A, B)[X / S] = Sigma(X, A[X / S], B)
  if true = true .
ceq Sigma(X, A, B)[X' / S] =
  Sigma(X, A[X' / S], B[X' / S])
  if X =/= X' .
eq pair(T1, T2, A)[V / S] = pair(T1[V / S], T2[V / S], A[V / S]) .
eq fst(T)[V / S] = fst(T[V / S]) .
eq snd(T)[V / S] = snd(T[V / S]) .
eq (A / R)[V / S] = (A[V / S]) / (R[V / S]) .
eq [A : R][ V / S] = [A[V / S] : R[V / S]] .
eq qelim(F, A, R)[V / S] = qelim(F[V / S], A[V / S], R[V / S]) .

eq Id(A, a, b)[V / S] = Id(A[V / S], a[V / S], b[V / S]) .
eq refl(a)[V / S] = refl(a[V / S]) .
eq qax(a, b, A, T)[V / S] = qax(a[V / S], b[V / S], A[V / S], T[V / S]) .
eq List[X / S] = List .
eq nil[X / S] = nil .
eq cons(T1, T2)[X / S] = cons(T1[X / S], T2[X / S]) .

op norm : Term -> Term .
op normApp : Term Term -> Term .
op normFst : Term -> Term .
op normSnd : Term -> Term .
op normNr : Term Term Term Term -> Term .
op normQe : Term Term Term -> Term .

eq norm(X) = X .
eq norm(Type) = Type .

eq norm(Pi(X : A, B)) = Pi(X : norm(A), norm(B)) .
eq norm((λ X : A . T)) = (λ X : norm(A) . norm(T)) .

eq norm(Id(A, a, b)) = Id(norm(A), norm(a), norm(b)) .
eq norm(refl(a)) = refl(norm(a)) .
eq norm(qax(a, b, A, T)) = qax(norm(a), norm(b), norm(A), norm(T)) .

eq norm(T1 ·T2) = normApp(norm(T1), norm(T2)) .

eq normApp((λ X : A . T), S) = norm(T[X / S]) .

eq normApp(T1, T2) = (T1 ·T2) [owise] .

eq norm(Nat) = Nat .
eq norm(zero) = zero .

eq norm(suc(T)) = suc(norm(T)) .

eq norm(natrec(A, T1, T2, T)) = normNr(norm(A), norm(T1), norm(T2), norm(T)) .
eq normNr(A, T1, T2, zero) = T1 .
eq normNr(A, T1, T2, suc(T)) = norm((T2 ·T) ·normNr(A, T1, T2, T)) .
eq normNr(A, T1, T2, T) = natrec(A, T1, T2, T) [owise] .
```

```
119
120   eq norm(Sigma(X, A, B)) = Sigma(X, norm(A), norm(B)) .
121   eq norm(pair(T1, T2, A)) = pair(norm(T1), norm(T2), norm(A)) .
122   eq norm(fst(T)) = normFst(norm(T)) .
123   eq normFst(pair(T1, T2, A)) = T1 .
124   eq normFst(T) = fst(T) [owise] .
125   eq norm(snd(T)) = normSnd(norm(T)) .
126   eq normSnd(pair(T1, T2, A)) = T2 .
127   eq normSnd(T) = snd(T) [owise] .
128
129   eq norm(A / R) = norm(A) / norm(R) .
130   eq norm([A : R]) = [norm(A) : norm(R)] .
131   eq norm(qelim(F, A, R)) = normQe(norm(F), norm(A), norm(R)) .
132   eq normQe(F, A, [T : QT]) = norm(F ·T) .
133   eq normQe(F, A, R) = qelim(F, A, R) [owise] .
134
135   eq norm(List) = List .
136   eq norm(nil) = nil .
137   eq norm(cons(T1, T2)) = cons(norm(T1), norm(T2)) .
138
139   op tyEq : Term Term -> Bool .
140   eq tyEq(A, B) = norm(A) == norm(B) .
141
142   sort Ctx .
143   op mt : -> Ctx [ctor] .
144   op _;_:_ : Ctx Var Term -> Ctx [ctor] .
145   var G : Ctx .
146
147   op lookup : Var Ctx -> [Term] .
148   eq lookup(X, G ; X : A) = A .
149   ceq lookup(X, G ; X' : A) = lookup(X, G) if X =/= X' .
150
151   op infer : Ctx Term -> [Term] .
152   op check : Ctx Term Term -> [Bool] .
153
154   eq infer(G, Type) = Type .
155
156   ceq infer(G, Id(A, T1, T2)) = Type
157     if Type := norm(infer(G, A))
158     /\ A := infer(G, T1)
159     /\ A := infer(G, T2) .
160
161   ceq infer(G, refl(T)) = Id(A, T, T) if A := infer(G, T) .
162
163   ceq infer(G, qax(T1, T2, A / R, T)) =
164     Id(A / R, [T1 : A / R], [T2 : A / R])
165     if true := check(G, T, (R ·T1) ·T2) .
166
167   eq infer(G, X) = lookup(X, G) .
168
169   ceq infer(G, Pi(V : A, B)) = Type
170     if Type := norm(infer(G, A))
171     /\ Type := norm(infer(G ; V : A, B)) .
172
173   ceq infer(G, (λ X : A . T)) = Pi(X : A, B)
174     if Type := norm(infer(G, A))
175     /\ B := infer(G ; X : A, T) .
176
177   ceq infer(G, T1 ·T2) = norm(B[X / T2])
178     if Pi(X : A, B) := norm(infer(G, T1))
179     /\ true := check(G, T2, A) .
180
181   eq infer(G, Nat) = Type .
182
183   eq infer(G, zero) = Nat .
184
185   ceq infer(G, A / R) = Type
186   if Type := norm(infer(G, A))
187   /\ true := check(G, R, Pi(a : A, Pi(b : A, Type))).
188
```

```
  ceq infer(G, [T : A / R]) = A / R
    if true := check(G, T, A) .

  ceq infer(G, qelim(F, Resp, Q)) = B
  if A / R := norm(infer(G, Q))
    /\ Pi(V : A , B) := norm(infer(G, F))
    /\ true := check(G, Resp,
        Pi(a : A , Pi(b : A ,
            Pi(p : (R ·a) · b ,
            Id(B, F ·a, F · b))))) .

  ceq infer(G, suc(T)) = Nat if true := check(G, T, Nat) .

  ceq infer(G, natrec(A, T1, T2, T)) = norm(A ·T)
    if true := check(G, T, Nat)
    /\ true := check(G, A, Pi(var("n") : Nat, Type))
    /\ true := check(G, T1, norm(A ·zero))
    /\ true := check(G, T2,
            Pi(var("k") : Nat,
            Pi(var("r") : (A ·var("k")),
                A · suc(var("k"))))) .

  ceq infer(G, Sigma(X, A, B)) = Type
    if Type := norm(infer(G, A))
    /\ Type := norm(infer(G ; X : A, B)) .

  ceq infer(G, pair(T1, T2, Sigma(X, A, B))) = Sigma(X, A, B)
    if true := check(G, T1, A)
    /\ true := check(G, T2, norm(B[X / T1])) .

  ceq infer(G, fst(T)) = A
    if Sigma(X, A, B) := norm(infer(G, T)) .

  ceq infer(G, snd(T)) = norm(B[X / fst(T)])
  if Sigma(X, A, B) := norm(infer(G, T)) .

  eq infer(G, List) = Type .
  eq infer(G, nil) = List .
  ceq infer(G, cons(T1, T2)) = List
  if true := check(G, T1, Nat)
  /\ true := check(G, T2, List) .

  eq check(G, T, A) = tyEq(infer(G, T), A) .
endfm
```

Apéndice B

Contribución a los Objetivos de Desarrollo Sostenible

Un trabajo teórico y fundacional como el presente no incide sobre los Objetivos de Desarrollo Sostenible de forma directa. Sin embargo, la tabla recoge el grado de relación con cada objetivo.

Objetivos de Desarrollo Sostenible	**Alto**	**Medio**	**Bajo**	**No procede**
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.		X		
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.				X
ODS 9. Industria, innovación e infraestructuras.	X			
ODS 10. Reducción de las desigualdades.				X
ODS 11. Ciudades y comunidades sostenibles.				X
ODS 12. Producción y consumo responsables.				X
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.			X	
ODS 17. Alianzas para lograr objetivos.				X

Cuadro B.1: Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.

La relación más apreciable es con el ODS 9, dado que el trabajo aporta a los fundamentos de los métodos formales, cuya finalidad es el software seguro. Guarda también un valor docente ODS 4, al ser un prototipo legible y ejecutable para estudiar teoría de tipos, y una conexión con el ODS 16 a través de la fiabilidad del software que sostiene servicios institucionales.